



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Светозар Пољак

**Реализација шаховске табле на бази
Bluetooth Low Energy протокола на
Raspberry Pi минирачунару**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026.



КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :	
Идентификациони број, ИБР :	
Тип документације, ТД :	Монографска публикација
Тип записа, ТЗ :	Текстуални штампани материјал
Врста рада, ВР :	Завршни рад
Аутор, АУ :	Светозар Пољак
Ментор, МН :	Др Иван Мезеи, редовни професор
Наслов рада, НР :	Реализација шаховске табле на бази Bluetooth Low Energy протокола на Raspberry Pi минарачунару
Језик публикације, ЈП :	Српски
Језик извода, ЈИ :	Српски
Земља публикавања, ЗП :	Република Србија
Уже географско подручје, УГП :	Аутономна Покрајина Војводина
Година, ГО :	2026.
Издавач, ИЗ :	Ауторски репринт
Место и адреса, МА :	Нови Сад, Трг Доситеја Обрадовића 6
Физички опис рада, ФО : (поглавља/страница/ цитата/табела/слика/графика/прилога)	6/45/31/5/37/0/0
Научна област, НО :	Електротехника и рачунарство
Научна дисциплина, НД :	Ембедед системи и алгоритми
Предметна одредница/Кључне речи, ПО :	Бежична комуникација, Bluetooth Low Energy, Raspberry Pi, Arduino Uno, шаховски програм
УДК	
Чува се, ЧУ :	Библиотека Факултета техничких наука, Нови Сад
Важна напомена, ВН :	
Извод, ИЗ :	У раду је описана реализација електронске шаховске табле засноване на дигиталним Hall сензорима. Положај шаховских фигура детектује се помоћу сензора распоређених по пољима табле, а добијено стање се Bluetooth Low Energy бежичном комуникацијом прослеђује Raspberry Pi рачунару. На рачунару је реализован графички кориснички интерфејс који омогућава приказ позиције, проверу валидности потеза и избор временског формата партије. Интеграцијом шаховског програма Stockfish омогућени су приказ евалуације позиције и предлагање најбољег потеза.
Датум прихватања теме, ДП :	21.9.2026.
Датум одбране, ДО :	22.9.2026.
Чланови комисије, КО :	Председник: Др Вук Врањковић, ванредни професор
	Члан: Др Предраг Теодоровић, ванредни професор
	Члан, ментор: Др Иван Мезеи, редовни професор
	Потпис ментора



KEY WORDS DOCUMENTATION

Accession number, ANO :			
Identification number, INO :			
Document type, DT :	Monographic publication		
Type of record, TR :	Textual printed document		
Contents code, CC :	Bachelor thesis		
Author, AU :	Svetozar Poljak		
Mentor, MN :	PhD Ivan Mezei, professor		
Title, TI :	Implementation of Chess Table on Raspberry Pi Minicomputer based on Bluetooth Low Energy Protocol		
Language of text, LT :	Serbian		
Language of abstract, LA :	Serbian		
Country of publication, CP :	Republic of Serbia		
Locality of publication, LP :	Autonomous Province of Vojvodina		
Publication year, PY :	2026.		
Publisher, PB :	Author's reprint		
Publication place, PP :	Novi Sad, Sq. Dositeja Obradovića 6		
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	6/45/31/5/37/0/0		
Scientific field, SF :	Electrical Engineering and Computer Science		
Scientific discipline, SD :	Embedded Systems and Algorithms		
Subject/Key words, S/KW :	Wireless communication, Bluetooth Low Energy, Raspberry Pi, Arduino Uno, chess engine		
UC			
Holding data, HD :	Library of the Faculty of Technical Sciences, Novi Sad		
Note, N :			
Abstract, AB :	This thesis describes the implementation of an electronic chessboard based on digital Hall-effect sensors. The positions of the chess pieces are detected using sensors distributed across the squares of the board, and the acquired board state is transmitted to a Raspberry Pi computer via Bluetooth Low Energy wireless communication. A graphical user interface is implemented on the computer, providing board position visualization, move validity checking, and selection of the time control for the game. Integration of the Stockfish chess engine enables position evaluation and best-move suggestions.		
Accepted by the Scientific Board on, ASB :	21.9.2026.		
Defended on, DE :	22.9.2026.		
Defended Board, DB :	President:	PhD Vuk Vranjković, associate professor	Menthor's sign
	Member:	PhD Predrag Teodorović, associate professor	
	Member, Mentor:	PhD Ivan Mezei, professor	



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

Број:

Датум:

ЗАДАТАК ЗА ЗАВРШНИ РАД

(Податке уноси предметни наставник - ментор)

Студијски програм:	Енергетика, електроника и телекомуникације		
Студент:	Светозар Пољак	Број индекса:	ЕЕ 35/2021
Степен и врста студија:	Основне академске студије		
Област:	Ембедед системи и алгоритми		
Ментор:	Др Иван Мезеи		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:			
- проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Реализација шаховске табле на бази Bluetooth Low Energy протокола на Raspberry Pi минирачунару

ТЕКСТ ЗАДАТКА:

У овом завршном раду потребно је урадити следеће:

- Упознати се са радом Hall сензора и BLE технологијом
- Упознати се са одговарајућим компонентама система за потребе играња шаха (нпр. Stockfish и сл.)
- Осмислити и пројектовати архитектуру шаховске табле и система за играње шаха
- Реализовати хардверски и софтверски део на Raspberry Pi минирачунару
- Тестирати реализован систем
- Дискутовати добијене резултате и написати завршни рад

Руководилац студијског програма:	Ментор рада:
Др Милан Сечујски	Др Иван Мезеи

Примерак за: ☐ - Студента; ☐ - Ментора

Образац **Q2.НА.04-03** - Издање 1

Изјава о академској честитости

Студент/киња: _____

Број индекса: _____

Студент/киња основних или мастер академских студија

Аутор/ка рада под називом:

Потписивањем изјављујем:

- да је рад искључиво резултат мог сопственог истраживачког рада;
- да сам рад и мишљења других аутора које сам користио/ла у овом раду назначио/ла или цитирао/ла и наведени у списку литературе/референци који су саставни део овог рада;
- да сам добио/ла све дозволе за коришћење ауторског дела који се у потпуности/целости уносе у предати рад и да сам то јасно навео/ла;
- да сам свестан/на да је плагијат коришћење туђих радова у било ком облику (као цитата, парафраза, слика, табела, дијаграма, дизајна, планова, фотографија, филма, музике, формула, веб сајтова, компјутерских програма и сл.) без навођења аутора или представљање туђих ауторских дела као мојих, кажњиво по закону (Закон о ауторском и сродним правима, Службени гласник Републике Србије, бр. 104/2009, 99/2011, 119/2012), као и других закона и одговарајућих аката Универзитета у Новом Саду;
- да сам свестан/на да плагијат укључује и представљање, употребу и дистрибуирање рада предавача или других студената као сопствених;
- да сам свестан/на последица које код доказаног плагијата могу проузроковати на предати рад и мој статус;
- да је електронска верзија рада идентична штампаном примерку и пристајем на његово објављивање под условима прописаним актима Универзитета.

У Новом Саду, _____

Потпис студента/киње



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

САДРЖАЈ

1. УВОД	1
2. ТЕОРИЈСКЕ ОСНОВЕ ХАРДВЕРСКИХ КОМПОНЕНТИ	2
2.1 <i>Raspberry Pi</i>	2
2.2 Дигитални <i>Hall</i> сензор <i>A3144</i>	3
2.3 <i>Arduino Uno</i>	3
2.4 <i>Bluetooth Low Energy</i> бежична технологија	5
2.4.1 Контролер	6
2.4.2 Хост-контролер интерфејс	7
2.4.3 Хост	7
2.4.3.1 <i>ATT</i>	7
2.4.3.2 <i>GATT</i>	8
2.4.3.3 <i>GAP</i>	9
2.5 <i>Bluetooth Low Energy</i> модул <i>AT-09</i>	10
3. ТЕОРИЈСКЕ ОСНОВЕ СОФТВЕРСКИХ КОМПОНЕНТИ	12
3.1 <i>FEN</i> нотација	12
3.2 <i>C++</i> библиотека <i>chess.h</i>	13
3.2.1 Класа <i>Board</i>	13
3.2.1.1 Класе <i>Piece</i> , <i>PieceType</i> , <i>Color</i> и <i>Square</i>	15
3.2.2 Класа <i>Move</i>	17
3.2.3 Структура <i>Movelist</i> и класа <i>movegen</i>	18
3.3 <i>Stockfish</i>	19
3.3.1 <i>UCI</i> протокол	19
3.4 <i>C</i> библиотека за рад са <i>Arduino USART</i> -ом	19
3.5 <i>Qt</i> радни оквир	20
4. РЕАЛИЗАЦИЈА РАДА	22
4.1 Хардверска реализација	22
4.1.1 Табла са сензорима	22
4.1.2 Повезивање <i>Arduino</i> развојног система и модула <i>AT-09</i>	26
4.2 Реализација програмске подршке	27
4.2.1 <i>Arduino</i> програм за обилазак поља табле	27
4.2.2 Програмска подршка за <i>Raspberry Pi</i>	30
4.2.2.1 Повезивање <i>AT-09</i> са <i>Raspberry Pi</i> -јем	30
4.2.2.2 Графички кориснички интерфејс	35
4.2.2.3 Обрада новог стања табле	38
4.2.2.4 Комуникација са <i>Stockfish</i> програмом	39
4.3 Систем у крајњем стадијуму израде пројекта	41
5. ЗАКЉУЧАК	42
6. ЛИТЕРАТУРА	43

ЛИСТА СЛИКА

Слика 2-1 Raspberry Pi 3 модел A+.....	2
Слика 2-2 Кућиште сензора A3144	3
Слика 2-3 Функционални блок дијаграм сензора A3144 [3]	3
Слика 2-4 Arduino Uno развојни систем	4
Слика 2-5 Bluetooth Low Energy протокол стек [7]	6
Слика 2-6 GATT хијерархија података [8].....	8
Слика 2-7 Изглед модула AT-09	10
Слика 3-1 Почетна позиција партије.....	12
Слика 3-2 Објекти <code>sender</code> и <code>receiver</code>	20
Слика 3-3 Објекти након позива методе <code>connect</code>	21
Слика 4-1 Архитектура система.....	22
Слика 4-2 Шема повезивања једног Hall сензора.....	23
Слика 4-3 Изглед модула (лево) и распоред прикључака на чипу CD74HC4067 тог модула (десно)	23
Слика 4-4 Повезивање једног сегмента са мултиплтксером и Arduino-ом	25
Слика 4-5 Распоред мултиплексера и повезивање са Arduino-ом	25
Слика 4-6 Шема повезивања Arduino развојног система и модула AT-09	26
Слика 4-7 Шема комплетне шаховске табле	26
Слика 4-8 Сегментација шаховске табле	27
Слика 4-9 Пресликавање индексирања спроведеног мултиплексерима у индексирање елемената низа <code>new_state</code>	28
Слика 4-10 Садржај низа <code>new_state</code> (десно) за приказану позицију (лево)	28
Слика 4-11 Алгоритам рада Arduino програма.....	29
Слика 4-12 lichess.org графички кориснички интерфејс [28].....	35
Слика 4-13 Распоред објеката корисничког интерфејса	36
Слика 4-14 Изглед графичког корисничког интерфејса	37
Слика 4-15 Мени за избор временског формата (лево), промоције белих фигура (средина) и промоције црних фигура (десно)	37
Слика 4-16 Систем у крајњем стадијуму израде пројекта	41
Слика 5-1 Две позиције једнаке окупираности поља фигурама	42

ЛИСТА ТАБЕЛА

Табела 2-1 Регистри модула USART0 и њихове најважније улоге	4
Табела 2-2 Пример неких модова и њима применљивих улога и процедура других уређаја [9]	10
Табела 3-1 Ознаке фигура у FEN нотацији	12
Табела 3-2 Ознаке рокада у FEN нотацији	13
Табела 4-1 Истинитосна таблица чипа CD74HC4067 [25].....	24

1. УВОД

У овом раду описана је шаховска табла која своју тренутну позицију фигура прослеђује *Raspberry Pi* рачунару. *Raspberry Pi* потом приказује позицију на монитору и проверава валидност потеза у оквиру графичког корисничког интерфејса (енгл. *Graphical User Interface – GUI*). Могуће је изабрати временски формат за партију, приказати евалуацију позиције и предложити најбољи потез коришћењем шаховског програма *Stockfish*.

Шаховска табла је реализована као мрежа дигиталних сензора базираних на Холовом ефекту. Сваком од поља табле је додељен један сензор. Свака фигура на свом дну има стални магнет, тако да табла детектује присуство фигура на својим пољима.

Идеја за рад је настала као унапређење пројекта на предмету Рачунарска електроника, где се стање табле слало краткоспојницима на улазно-излазни порт опште намене *Raspberry Pi* рачунара. С обзиром на то да је фокус пројекта био на упознавању са *Raspberry Pi* рачунаром и на развоју софтвера, овакво решење је било прихватљиво.

На предлог професора др Ивана Мезеија уведена је бежична комуникација. За оригинални пројекат је коришћен *Raspberry Pi 3* модел *A+*, који подржава *Bluetooth Low Energy* бежичну технологију, стога се она користи у овом решењу.

Графичком корисничком интерфејсу је посвећено доста пажње. Корисници преко фигура, тастатуре и миша комуницирају са графичким корисничким интерфејсом на интуитиван начин. Шаховска партија се фигурама одвија на уобичајен начин, а употреба тастатуре и миша је идентична као код стоних или лаптоп рачунара. Ово је управо оно што се очекује од модерних система. За ефикасно руковање није потребно додатно познавање архитектуре система и комуникационих протокола, већ само правила шаха.

У поглављу 2 ТЕОРИЈСКЕ ОСНОВЕ ХАРДВЕРСКИХ КОМПОНЕНТИ дат је преглед основних теоријских концепата хардверских компоненти које су искоришћене за реализацију хардвера. Поглавље 3 ТЕОРИЈСКЕ ОСНОВЕ СОФТВЕРСКИХ КОМПОНЕНТИ описује теоријске основе библиотека и програма који се користе за реализацију софтвера.

Поглавље 4 РЕАЛИЗАЦИЈА РАДА објашњава конкретно решење са софтверске и хардверске тачке гледишта. У првом потпоглављу, 4.1 Хардверска реализација, описује се начин на који је табла направљена. У другом потпоглављу, 4.2 Реализација програмске подршке, објашњава се програм за скенирање шаховске табле, употреба бежичне комуникације, интеграција шаховског програма и графички кориснички интерфејс. Последње поглавље, 5 ЗАКЉУЧАК, даје осврт на цео пројекат и пружа неке могуће препоруке за побољшање система.

2. ТЕОРИЈСКЕ ОСНОВЕ ХАРДВЕРСКИХ КОМПОНЕНТИ

У овом поглављу наглашене су само најважније особине хардверских компоненти које се користе у систему.

2.1 Raspberry Pi

Raspberry Pi је ембедед рачунар реализован на једној штампаној плочици (енгл. *single-board computer*). Сви модели поседују неку верзију *Broadcom* система на чипу (енгл. *system on chip*). На чипу се налазе *ARM* процесор, графички процесор и *RAM* (енгл. *Random-Access Memory*) меморија [1]. *RAM* меморија је дељена са графичким процесором. Њен капацитет зависи од модела и креће се у опсегу од 256 MB до чак 16 GB на новијим моделима [2]. Ови рачунари се напајају номиналним напоном од 5 V (5.1 V код новијих модела) преко *microUSB* или *USB-C* порта. Пошто сваки модел поседује и улазно-излазни порт опште намене, напајање је могуће и преко њега.

Интерфејс *Raspberry Pi*-ја такође зависи од конкретног модела, али већина поседује *USB 2.0* порт и *HDMI* или *mini-HDMI* видео излаз. Оперативни систем, програми и подаци се смештају на *SD* или *microSD* картицу, за коју постоји слот на плочици. Од рачунара *Raspberry Pi 3* модел *B* уводи се *Bluetooth 4.1* бежична комуникација [1].

Конкретан модел који се користи у систему је *Raspberry Pi 3* модел *A+*, приказан на слици 2-1.



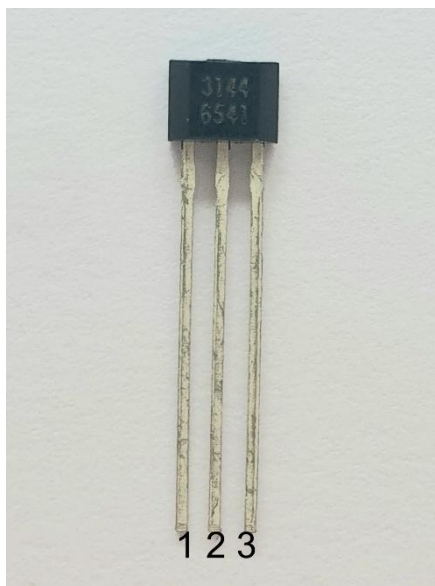
Слика 2-1 Raspberry Pi 3 модел A+

Модел *A* је јефтинија варијанта, са мање *RAM* меморије, *USB* портова и без *Ethernet* порта, за разлику од модела *B* [2].

Raspberry Pi 3 модел *A+* на себи има *Broadcom BCM2837B0* систем на чипу. На њему су 64-битни *ARM* процесор *Cortex-A53 (ARMv8)* и *RAM* меморија капацитета 512 MB, *SDRAM* (енгл. *Synchronous Dynamic Random-Access Memory*) типа технолошке израде [2]. Рачунар се напаја номиналним напоном од 5 V и струјом од 2.5 A преко *microUSB* порта. Поседује један *USB 2.0* порт, *HDMI* видео излаз и *microSD* слот. Бежичну комуникацију обезбеђују двоопсежни *Wi-Fi (2.4 GHz и 5 GHz)* и *Bluetooth 4.2*, који може радити и у *Bluetooth Low Energy* режиму. Ова два протокола деле исти чип и антену на плочици.

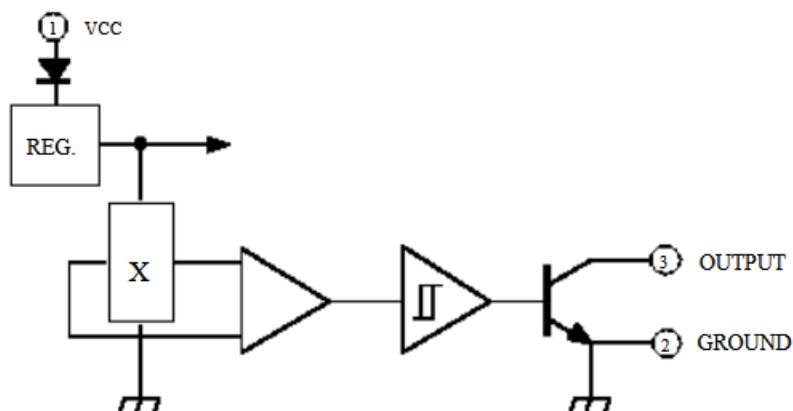
2.2 Дигитални *Hall* сензор A3144

Дигитални *Hall* сензор A3144 је монолитно интегрисано коло које ради на принципу Холовог ефекта. На слици 2-2 је приказано кућиште овог сензора са његове брендиране стране.



Слика 2-2 Кућиште сензора A3144

Прикључци за напајање, уземљење и дигитални излаз су редом означени бројевима 1-3. Функционални блок дијаграм сензора је приказан на слици 2-3 [3].



Слика 2-3 Функционални блок дијаграм сензора A3144 [3]

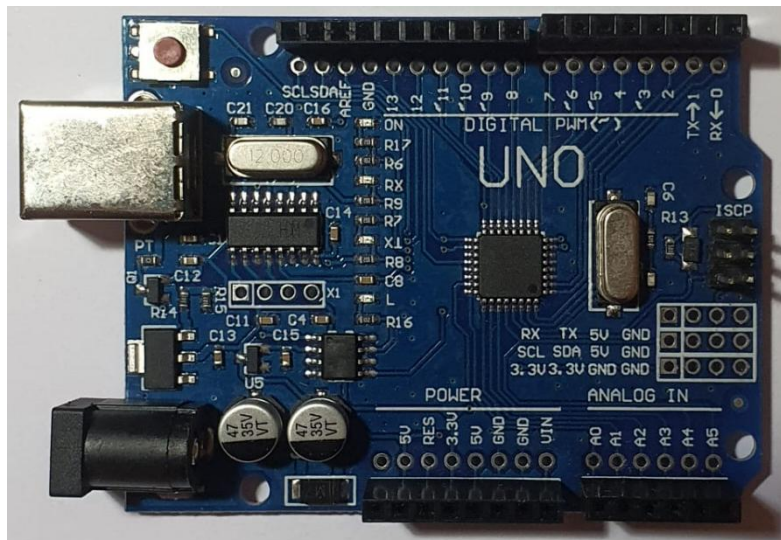
Регулатор напона (на слици 2-3 означен блоком *REG.*) обезбеђује напон напајања у опсегу од 4.5 V до 24 V. Диода обезбеђује заштиту од инверзног поларисања напона напајања. Излаз Холовог генератора напона (на слици 2-3 означеног блоком *X*) се доводи на улаз појачавача малих сигнала. У колу је присутан Шмитов тригер који успоставља стабилан дигитални сигнал и отклања шум. Његов излаз се доводи на отворени колектор (енгл. *open-collector*), чија је струја понирања (енгл. *sink*) до 24 mA. Уз одговарајући *pull-up* отпорник може да се користи са биполарном или *CMOS* логиком [3].

Ово је униполарни сензор који реагује на присуство јужног пола магнета. Због отвореног колектора, на свом излазу даје низак напон уколико му се приближи јужни пол магнета.

2.3 Arduino Uno

Arduino Uno је развојни систем базиран на *RISC* микроконтролеру *ATmega328P*. Микроконтролер је 8-битни и ради на фреквенцији од 16 MHz. Меморије за програме и

податке су одвојене. Користи се 32 kB флеш меморија за програме, са присутним *bootloader* програмом који заузима 2 kB. Меморија за податке је *SRAM* (енгл. *Static Random Access Memory*) капацитета 2 kB. *ATmega328P* има и 1 kB *EEPROM* (енгл. *Electrically-Erasible/Programmable Read-Only Memory*) меморије. На слици 2-4 је приказана верзија *Arduino Uno* развојног система који на себи има *ATmega328P* у *TQFP* (енгл. *thin quad flat pack*) кућишту [4].



Слика 2-4 *Arduino Uno* развојни систем

Већина *ATmega328P* прикључака припадају улазно-излазним портovima *Port B*, *Port C* и *Port D*. За контролу смера података, читавања улазних вредности и слања излазних вредности на прикључцима се користе регистри *DDRx*, *PINx* и *PORTx*, где *x* замењује ознаку порта (*B*, *C* или *D*). Бити регистра *DDRx* одређују смер података на прикључцима порта *x*, односно да ли су они улазни или излазни. У *PINx* регистру се налази улазна вредност порта *x*, а у *PORTx* регистру се налази вредност која се шаље на прикључке порта *x* као излазна вредност [4].

RS-232 серијска комуникација на овом микроконтролеру је имплементирана модулом *USART0* (енгл. *Universal Synchronous/Asynchronous Receiver/Transmitter 0*). Регистри модула *USART0* и најважније улоге за овај рад су излистане у табели 2-1 [5].

Табела 2-1 Регистри модула *USART0* и њихове најважније улоге

Регистар	Улога
<i>UDR0</i>	Улазно-излазни регистар података 0 (енгл. <i>USART I/O DATA REGISTER 0</i>): - Вредност уписана у <i>UDR0</i> регистар се прослеђује у предајни бафер <i>TXB</i>. - Вредност прочитана из <i>UDR0</i> регистра је вредност прочитана из пријемног бафера <i>RXB</i>.
<i>UCSR0A</i>	Контролни и статусни регистар 0 А (енгл. <i>USART CONTROL AND STATUS REGISTER 0 A</i>): - Пружа информације да ли у пријемном баферу постоји непровучан податак. - Указује да ли је послат целокупан садржај предајног регистра и да не постоји нов податак у предајном баферу. - Указује да ли је предајни бафер спреман за пријем новог податка за слање.
<i>UCSR0B</i>	Контролни и статусни регистар 0 В (енгл. <i>USART CONTROL AND STATUS REGISTER 0 B</i>):

	- Омогућује укључивање 3 различита прекида: прекид када у пријемном баферу постоји непрочитан податак, прекид када је послат целокупан садржај предајног бафера и прекид када је предајни бафер спреман за пријем новог податка за слање. - Дозвола рада пријемника и у том случају модул <i>USART0</i> преузима контролу над прикључком 0 порта <i>Port D (PD0)</i>. - Дозвола рада предајника и у том случају модул <i>USART0</i> преузима контролу над прикључком 1 порта <i>Port D (PD1)</i>.
UCSR0C	Контролни и статусни регистар 0 C (енгл. <i>USART CONTROL AND STATUS REGISTER 0 C</i>): - Одређује један од 3 режима рада: асинхрони <i>USART</i>, синхрони <i>USART</i> или мастер <i>SPI</i>. - Омогућава конфигурисање паритета – да ли се користи и да ли је парни или непарни. - Омогућава конфигурисање броја бита података приликом серијског преноса. - Омогућава конфигурисање поларитета такт сигнала, што је од значаја само у синхронном <i>USART</i> режиму рада.
UBRR0L и UBRR0H	То су регистри за конфигурисање брзине серијске комуникације (енгл. <i>USART BAUD RATE REGISTERS</i>).

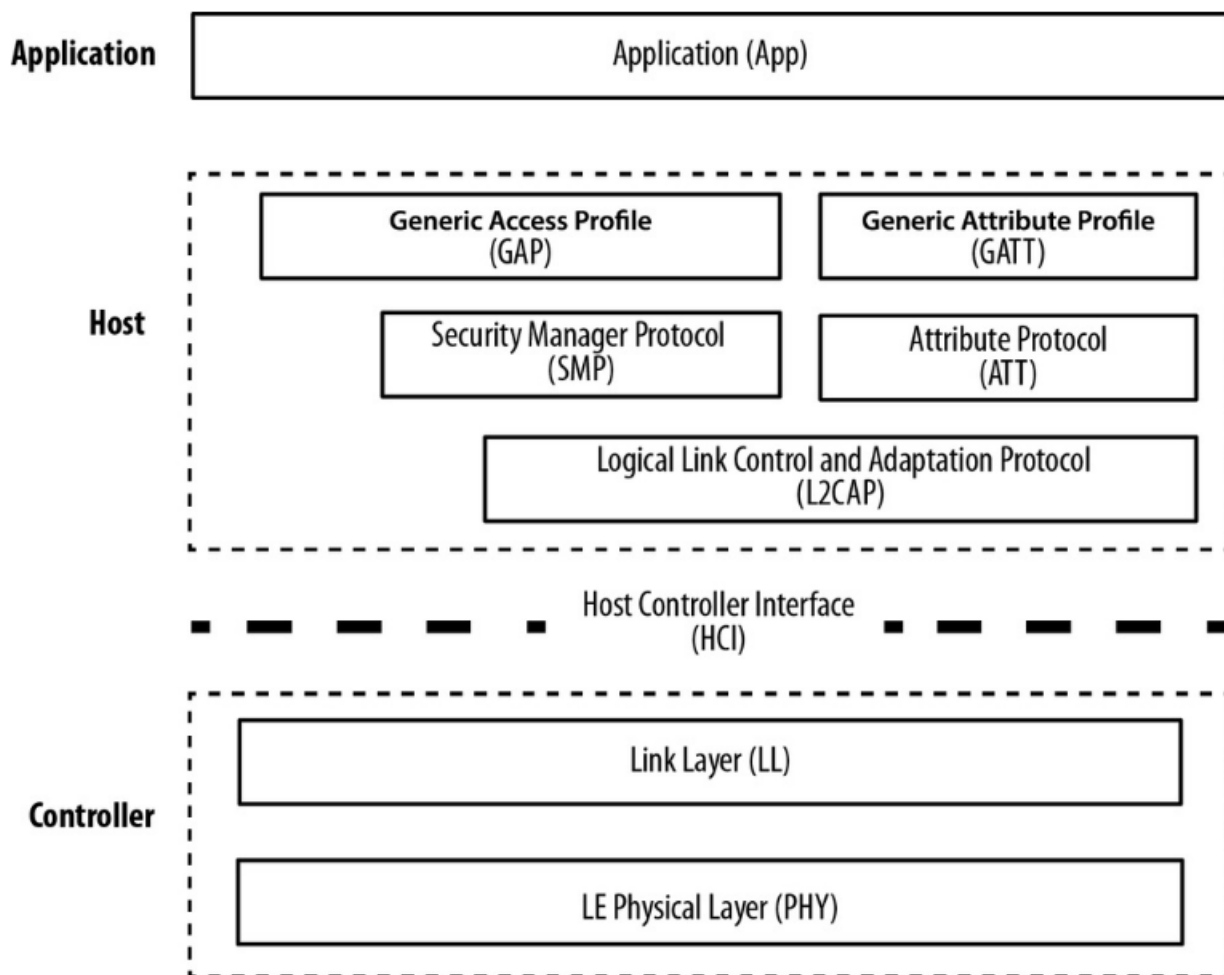
Брзина серијске комуникације се изражава у броју бита по секунди (bit/s). Регистри *UBRR0L* и *UBRR0H* из табеле 2-1 се користе заједно да дефинишу коришћену брзину серијске комуникације. Нижа четири бита *UBRR0H* регистра са *UBRR0L* регистром формирају 12-битну вредност *UBRR0*. Нека је *BAUD* брзина жељене серијске комуникације. Уколико је F_{osc} учестаност такта микроконтролера, вредност *UBRR0* се израчунава по једначини 1.

$$UBRR0 = \frac{F_{osc}}{16 \cdot BAUD} - 1 \quad (1)$$

2.4 Bluetooth Low Energy бежична технологија

Bluetooth (познат и као *classic Bluetooth*) је бежична технологија којом се подаци преносе између уређаја путем радио таласа у фреквенцијском опсегу од 2.402 GHz до 2.48 GHz. Неки од примера коришћења *Bluetooth* технологије би били бежично повезивање мобилних телефона и лаптопова, бежичних слушалица, паметних сатова, аута и сл. Током година, развојем технологије и повећавањем броја могућих примена, јавља се потреба за бржим преносом већег броја података, самим тим и бржим радио уређајима [6].

Bluetooth Low Energy – *BLE* се уводи у верзији *Bluetooth 4.0* и фокусира се на другачијем тржишту уређаја. Овде се приоритет ставља на оптимизацију у смислу уштеде енергије. Схвата се да *classic Bluetooth* не може да задовољи потребе за малом потрошњом јефтиних уређаја пуњивих помоћу батерија [6]. Циљ је развој јефтиног и једноставног радио стандарда који троши што је могуће мање енергије, на рачун и брзине преноса и величине пакета података. *Bluetooth Low Energy* протокол стек је приказан на слици 2-5 [7].



Слика 2-5 Bluetooth Low Energy протокол стек [7]

Bluetooth Low Energy протокол стек се састоји из три дела: апликације, хоста и контролера. Апликација је највиши слој протокол стека и имплементира у софтверу неки специфичан случај коришћења интерагујући са нижим слојевима. Хост се састоји из:

- профила за генеричан приступ (енгл. *Generic Access Profile – GAP*)
- профила генеричког атрибута (енгл. *Generic Attribute Profile – GATT*)
- протокола атрибута (енгл. *Attribute Protocol – ATT*)
- протокола менаџера безбедности (енгл. *Security Manager Protocol – SMP*)
- протокола контроле логичког повезивања и адаптације (енгл. *Logical Link Control and Adaptation Protocol – L2CAP*)
- хост дела хост-контролер интерфејса (енгл. *Host Controller Interface – HCI*).

Генерално, протоколи имплементирају различите аспекте комуникације по спецификацији, док профили дефинишу како би се протоколи требали користити да се оствари генерички или специфичан циљ [7]. Контролер се састоји из контролер дела *HCI*-ја, линк слоја (енгл. *Link Layer – LL*) и физичког слоја (енгл. *Physical Layer – PHY*).

2.4.1 Контролер

Унутар физичког слоја се налази радио који шаље и прима радио таласе из 2.4 GHz *ISM* опсега. Назив ове групе фреквенција је скраћеница њене области примене – то је нелиценциран опсег који се користи у индустрији, науци и медицини (енгл. *Industrial, Medical and Scientific – ISM*). Bluetooth није једина технологија која користи овај опсег и као решење проблема интерференције опсег је подељен на 40 канала, од 2.4 GHz до 2.4835 GHz. Радио користи ову поделу и мења канале по унапред познатом шаблону користећи технику *frequency hopping spread spectrum*.

Линк слој је комбинација наменског хардвера и софтвера. Хардвер је задужен за временски критичне аспекте протокол стека, а софтверска страна управља стањима повезивања радија. С тим у вези, линк слој дефинише улоге оглашавачког уређаја (енгл. *advertiser*), скенера (енгл. *scanner*), мастера (енгл. *master*) и слејва (енгл. *slave*). Оглашавач шаље оглашавачке пакете, а скенер трага за оглашавачким пакетима. Мастер иницира везу и њоме руководи када се успостави, док слејв прихвата везу и касније прати мастеров тајминг [7].

2.4.2 Хост-контролер интерфејс

HCI је граница између дела протокол стека који извршава временски критичне операције на наменском хардверу и хост дела протокола који може да се извршава на обичном процесору. На пример, мобилни телефони интегришу *Bluetooth* тако што уграде модул који имплементира контролер протокол стека, а за горње слојеве користе већ постојећи процесор. Комуникација између њих је онда дефинисана делом спецификације који описује *HCI*. Са друге стране, ембедед уређаји користе систем на чипу где су сва три дела протокол стека имплементирана на једном чипу.

HCI је сет команди и догађаја којим се дефинише како хост и контролер интерагују. Дефинише и формат пакета података који се размењују између хоста и контролера, скуп правила којима се спроводи контрола и друге процедуре [7]. Комуникација хоста и контролера се обично извршава преко серијског интерфејса попут *USB*-а или *UART*-а.

2.4.3 Хост

L2CAP мултиплексира протоколе виших слојева протокол стека у пакете разумљиве нижим слојевима и обрнуто. Задужен је за рутирање два главна протокола – протокола атрибута (*ATT*) и протокола менаџера безбедности (*SMP*).

Менаџер безбедности (енгл. *Security Manager* – *SM*) је уједно и протокол и скуп безбедносних алгоритама који гарантује упареним уређајима безбедну комуникацију енкриптованом везом, заштиту од малициозних уређаја и сл [7].

2.4.3.1 *ATT*

За *ATT* протокол, *BLE* уређаји могу да буду сервери, клијенти или оба истовремено. Подаци су организовани у адресабилне структуре које се називају атрибути. Атрибути су сачињени из:

- хендл атрибута (енгл. *attribute handle*) – 16-битни идентификатор који се користи за приступ вредности атрибута
- универзално јединственог идентификатора (енгл. *Universally Unique Identifier* – *UUID*) – 128-битни идентификатор који специфицира тип и природу вредности атрибута
- вредност атрибута – податак од значаја, максималне величине 512 бајтова
- додатних дозвола [7].

Углавном клијент иницира комуникацију и захтева приступ атрибутима сервера, док сервер по потреби одговара. Постоји скуп дозвољених *ATT* операција, које се могу обављати преко *ATT* протокола:

- управљање грешком насталом током извођења претходне операције
- конфигурација сервера
- операције које клијент користи да добије информације о структури серверових атрибута
- читање вредности атрибута преко хендла атрибута или *UUID*-ја
- упис вредности атрибута, са или без очекивања одговора сервера; постоји и посебна варијанта тзв. заказаног уписа (енгл. *queued write*) у случају потребе за уписом веће количине података од оне која може да стане у један пакет

- операције инициране од стране сервера – сервер обавештава клијента о промени вредности атрибута идентификованог преко свог хендла [7].

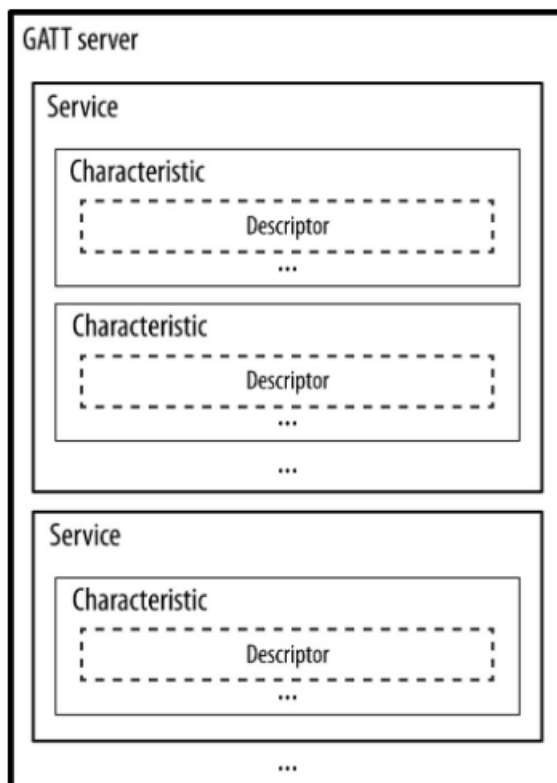
Хендл чини атрибут адресабилним и гарантује се да се хендл не мења. Пошто би 128-битна *UUID* вредност заузела велики део пакета нижих нивоа (*LL* пакет је величине 27 бајтова), *BLE* спецификација уводи и посебне 16-битне и 32-битне *UUID* вредности. На основу ових вредности се реконструише 128-битна вредност тако што увек заузима формат `xxxxxxxx-0000-1000-8000-00805F9B34FB`. Прва 32 бита означена са *x* представљају полазне 16-битне или 32-битне *UUID* вредности, при чему се 16-битној вредности додају водеће нуле до 32-битне вредности [8].

Додатне дозволе у атрибуту су метаподаци који указују које *ATT* операције су дозвољене а које нису. Постоје дозволе приступа клијента вредности атрибута, дефинише се који ниво енкрипције се користи за приступ атрибуту и дефинише се да ли је за приступ атрибуту потребно одобрење корисника. Само ако ове дозволе одобравају, клијент може слободно приступати (и/или модификовати) вредности атрибута [8].

2.4.3.2 GATT

Подаци битни за апликацију и корисника се пакују у структуре и шаљу у складу са *GATT* правилима. *GATT* користи *ATT* протокол за размену података међу уређајима. Уређаји узимају улоге клијента и сервера и одговарају клијенту и серверу у *ATT* протоколу. Клијент шаље захтеве серверу и добија одговоре од њега. Сервер складишти податке битне кориснику и чини их приступачним за клијента. Сервер може бити конфигурисан и да иницира нека обавештења.

GATT користи атрибуте да формира хијерархију података приказану на слици 2-6 [8].



Слика 2-6 *GATT* хијерархија података [8]

Ову хијерархију поштују сви *BLE* уређаји, стога су сви атрибути смештени унутар ње. Атрибути унутар *GATT* сервера су организовани у сервисе (енгл. *service*). Сви атрибути у сервису чине дефиницију сервиса. Почетни атрибут у сервису означава декларацију сервиса. Организација декларације сервиса одговара стандардној организацији атрибута (видети 2.4.3.1 *ATT*) је следећа:

- одговарајући хендл атрибута

- тип који је 16-битни *UUID* вредности 0x2800, ређе 0x2801; предефинисан је стандардом и резервисан за декларацију сервиса
- вредност атрибута је *UUID* сервиса који ова декларација представља, дужине 16, 32 или 128 бита
- дозвола искључиво за читање.

Сервис може имати референце на друге сервисе помоћу *include* дефиниција. Унутар *include* дефиниција постоји један атрибут који се зове *include* декларација. Његова организација обухвата:

- одговарајући хендл атрибута
- стандардни предефинисани *UUID* вредности 0x2802
- вредност атрибута – укључује хендл почетка референцираног сервиса, хендл краја тог сервиса и његов *UUID*
- дозвола искључиво за читање.

Овим механизмом се избегава дуплирање података у *GATT* серверу и читава структура се поједностављује [8].

Карактеристике су део сервиса и садрже податке од значаја. Карактеристика је сачињена од барем два атрибута, декларације карактеристике и вредности карактеристике.

Декларација карактеристике је атрибут чији је тип стандардни предефинисани *UUID* вредности 0x2803 који означава почетак карактеристике. Вредност атрибута садржи својства карактеристике (енгл. *characteristics properties*) – бит-поље које излистава дозвољене операције над карактеристиком. Неки од бита пружају информације о томе да ли је дозвољено емитовање вредности карактеристике, да ли су дозвољена *ATT* читања, уписи и сл. Вредност атрибута садржи и хендл атрибута у којем се налази вредност карактеристике и *UUID* карактеристике. Дозвољено је искључиво читање декларације.

Атрибут који представља вредност карактеристике има тип (*UUID*) који се налази у вредности декларације карактеристике. Вредност овог атрибута је информација коју је потребно пренети између два *BLE* уређаја – читавање сензора, притисак тастера тастатуре и сл.

Додатно, у карактеристици могу постојати и дескриптори који пружају информације које се надовезују на оне из декларације карактеристике. Заједно сви ови атрибути чине дефиницију карактеристике [8].

2.4.3.3 *GAP*

Четири основна аспекта интеракције међу уређајима које *GAP* дефинише су:

1. улоге уређаја (енгл. *roles*)
2. модови уређаја (енгл. *modes*)
3. процедуре (енгл. *procedures*)
4. безбедносни аспект (енгл. *security*).

Улоге уређаја које *GAP* дефинише су емитер (енгл. *broadcaster*), посматрач (енгл. *observer*), централни уређај (енгл. *central*) и периферни уређај (енгл. *peripheral*). Емитери шаљу податке у тзв. оглашавачким пакетима. Пакети су доступни свим уређајима који слушају. Емитери користе оглашавачку улогу линк слоја протокол стека (*LL*). Посматрач слуша оглашавачке податке уређаја са улогом емитера. Посматрачи користе *LL* улогу скенера. Централни уређај је способан да успоставља везу са више уређаја и увек је иницијатор повезивања. Одговара улози мастера у *LL*. Периферни уређаји оглашавачким пакетима омогућавају централним уређајима да их пронађу и да успоставе везу са њима. Одговарају улози слејва у *LL* [9].

Мод је стање у којем се уређај мора наћи да би извршио неки циљ или другом уређају омогућио да изврши одговарајућу процедуру. Процедуре су секвенце акција које уређај

мора извршити да би се циљ постигао. Додатно, од другог уређаја са којим интерагује се захтева да буде у одговарајућем моду, стога су мод и процедура уско повезани. Табела 2-2 даје пример ове повезаности [9].

Табела 2-2 Пример неких модова и њима применљивих улога и процедура других уређаја [9]

Mode	Applicable Role(s)	Applicable Peer Procedure(s)
Broadcast	Broadcaster	Observation
Non-discoverable	Peripheral	N/A
Limited discoverable	Peripheral	Limited and General discovery
General discoverable	Peripheral	General discovery
Non-connectable	Peripheral, broadcaster, observer	N/A
Any connectable	Peripheral	Any connection establishment

На пример, из првог реда табеле 2-2 се види да уколико је уређај у моду емитовања (енгл. *Broadcast*) улога коју може имати у мрежи је улога емитера, а други уређаји могу спровести процедуру посматрања (енгл. *Observation*).

Безбедносни аспект се надовезује на *SM* дефинишући модове и процедуре који специфицирају како уређаји у комуникацији постављају одређени ниво безбедности за пренос података и како се тај ниво спроводи. Дефинишу се и додатне опције невезане за модове и процедуре [9].

2.5 Bluetooth Low Energy модул AT-09

Модул *AT-09* је базиран на *Texas Instruments* систему на чипу *CC2541*. На чипу се налазе 2.4-GHz трансивер, микроконтролер, до 256 KB програмабилне меморије, 8 KB *RAM* меморије и периферијски модули [10].

Модул *AT-09* је производ компаније Болутек, која уједно пружа и фирмвер за *CC2541*. Стога је у питању тзв. *Single-Device* конфигурација, где је комплетан протокол стек имплементиран на чипу [10]. Изглед плочице је приказан на слици 2-7.



Слика 2-7 Изглед модула *AT-09*

Модул се састоји од мање штампане плочице са *CC2541* чипом, која је залемљена на већу *ZS-040* плочицу. Изворну детаљну документацију о самом модулу је тешко наћи, постоје Болутек документације о формату команди које се користе за конфигурацију модула – *AT* команде и кратка документација мање штампане плочице са *CC2541* чипом [11], [12]. Неке недостајуће информације о модулу су допуњене сајтовима програмера који су имали искуства са модулом и сајтовима за продају електронске опреме [13]-[16].

За напајање се користи прикључак *VCC* приказан на слици 2-7, док се прикључак *GND* користи за масу. Прикључци *RXD* и *TXD* реализују *UART* сигнале, док *GND* представља заједничку референтну тачку интерфејса. *AT-09* користи *UART* интерфејс за пријем *AT* команди и размену података. Када се модул повеже са другим уређајем који има *UART* интерфејс, ови прикључци се користе као *full-duplex* серијски порт [15]. Подразумевана брзина серијске комуникације је 9600 bit/s [13], [16]. *VCC* прикључак очекује напон у опсегу од 3.6 V до 6 V, док *RXD* користи логичке нивое од 3.3 V [15], [16].

Фирмвер који се користи је Болутеков *Firmware V3.0.6, Bluetooth V4.0 LE*. При паљењу, модул је у *AT* режиму (назива се и командни режим) где са *UART* интерфејса прихвата *AT* команде [16]. *AT* команде користе посебну синтаксу да конфигуришу модул. Команде почињу са *AT*, а обавезно се завршавају са `\r\n` [12].

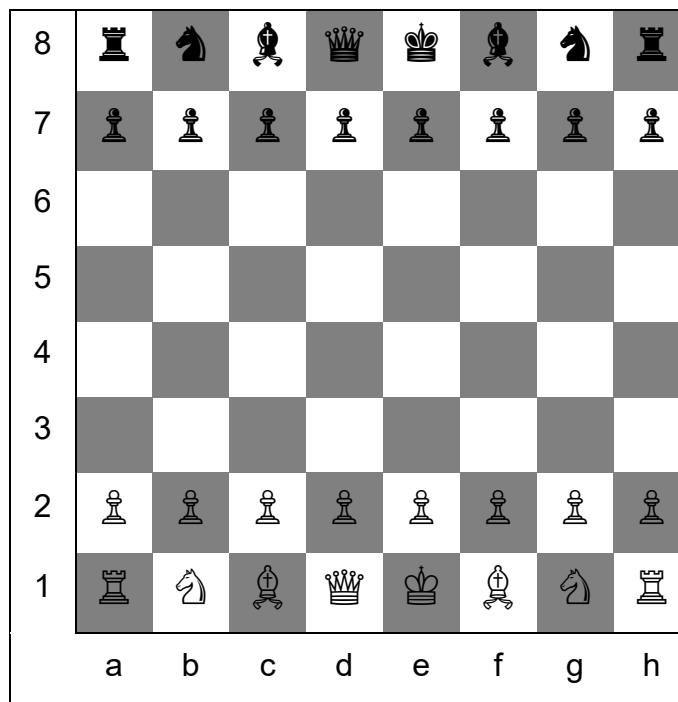
Модул је видљив другим *BLE* уређајима као *BT05* [16]. *UUID* сервиса има вредност 0xFFE0, док *UUID* карактеристике има вредност 0xFFE1. За индикацију стања везе може се користити *LED* диода на модулу, која трепери док није успостављена веза, а након успостављања везе непрекидно светли [11]. Успостављањем везе са другим *BLE* уређајем модул прелази у транспарентни режим (назива се и режим података), у коме се подаци примљени преко *UART* интерфејса прослеђују преко *BLE* везе, док се подаци примљени преко *BLE* везе прослеђују ка *UART* интерфејсу [13], [16]. Подразумевана улога модула је улога слејва, односно периферног уређаја у *GAP* смислу. Модул је могуће конфигурисати и као мастера одговарајућом *AT* командом [14].

3. ТЕОРИЈСКЕ ОСНОВЕ СОФТВЕРСКИХ КОМПОНЕНТИ

У наставку се објашњавају теоријске основе програмске подршке. Сматра се да се познају правила шаха.

3.1 FEN нотација

FEN је скраћеница за *Forsyth-Edwards* нотацију која представља стандардну нотацију за опис позиције шаховске партије. Овом нотацијом се комплетна позиција описује једним стрингом ASCII (енгл. *American Standard Code for Information Interchange*) карактера [17]. Начин на који FEN представља позицију је објашњен на примеру почетне позиције, приказане на слици 3-1.



Слика 3-1 Почетна позиција партије

Табла се означава стандардном алгебарском нотацијом [18]. Стање шаховске табле у FEN нотацији се састоји из следећих сегмената:

1. поља заузета фигурама и празна поља присутна на табли
2. ко је следећи на потезу – играч са белим или црним фигурама
3. рокаде које су доступне играчима
4. потенцијално одредишно поље за *en passant* заробљавање фигуре
5. број полупотеза
6. број целих потеза.

Први сегмент нотације описује попуњеност поља фигурама. Поља табле са слике 3-1 се обилазе по редовима, од осмог до првог. У сваком реду се пролази кроз сваку колону, од *a* до *h*. Уколико се наиђе на црну фигуру, она се представља малим словима абетеде, а уколико се наиђе на белу фигуру она се представља великим словима абетеде, као што је приказано у табели 3-1. Уколико се наиђе на празно поље, пребројава се број празних поља до следеће фигуре у том реду. Нотације редова се одвајају знаком / [17].

Табела 3-1 Ознаке фигура у FEN нотацији

фигура	ознака	фигура	Ознака
бели пешак	<i>P</i>	црни пешак	<i>P</i>

бели коњ	<i>N</i>	црни коњ	<i>N</i>
бели ловац	<i>B</i>	црни ловац	<i>B</i>
бели топ	<i>R</i>	црни топ	<i>R</i>
бела краљица	<i>Q</i>	црни краљица	<i>Q</i>
бели краљ	<i>K</i>	црни краљ	<i>K</i>

У другом сегменту нотације, ознака да је бели на потезу је *w*, а да је црни на потезу је *b*. У трећем сегменту се наводе све доступне рокаде. Рокада је доступна уколико се краљ и одговарајући топ још нису померили. У табели 3-2 су приказане ознаке рокада које се користе. Прво се излиставају доступне рокаде играча са белим фигурама, па потом играча са црним фигурама. Уколико ниједна страна нема право на ниједну рокаду, то се означава симболом – [17].

Табела 3-2 Ознаке рокада у FEN нотацији

	мала рокада	велика рокада
бели	<i>K</i>	<i>Q</i>
црни	<i>k</i>	<i>q</i>

Четврти сегмент је ознака поља које је непосредно иза пешака који се управо померио за два места. Тај пешак је сада потенцијална мета за *en passant* заробљавање. Поље ће се у тексту надаље означавати ознаком колоне поља (*a-h*) у комбинацији са ознаком реда поља (1-8), на пример *e3*. Уколико у тренутној позицији нема потенцијалних мета за *en passant* заробљавање, овај сегмент се означава симболом – [17].

Последња два сегмента су бројеви полупотеза и целих потеза. Број полупотеза је број потеза протеклих од последњег заробљавања фигуре или напредовања пешака и користан је податак за реми правило 50 потеза. Када број полупотеза достигне 100, оба играча су одиграла 50 потеза и партија завршава ремијем. Број целих потеза се инкрементује сваки пут када играч са црним фигурама одигра потез [17]. Узевши све претходно у обзир, FEN стринг почетне позиције приказане на слици 3-1 је *rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1* [19].

3.2 C++ библиотека `chess.h`

Библиотека `chess.h` се састоји из само једног заглавља (енгл. *single header library*) за чије је коришћење довољно да се оно укључи у изворни код помоћу `#include` претпроцесорске директиве. Класе, методе и набројиви типови који се користе као API (енгл. *Application Programming Interface*) се налазе у именском простору `chess` [19].

3.2.1 Класа `Board`

Класа `Board` представља тренутну позицију шаховске табле. У наставку су излистане методе ове класе које се користе у раду. Декларација конструктора је дата у код листингу 3-1 [19].

Код листинг 3-1 Декларација конструктора класе `Board`

```
explicit Board(std::string_view fen = constants::STARTPOS, bool chess960 = false);
```

Подразумеване вредност параметра `fen` је константа `STARTPOS` која представља FEN стринг почетне позиције, приказане на слици 3-1. У овом раду се неће разматрати *Fisher Random* шах, те се и за други параметар користи подразумевана вредност.

Пример методе за мењање позиције табле је метода за одигравање потеза – `makeMove`. Декларација методе `makeMove` је приказана у код листингу [19].

Код листинг 3-2 Декларација методе `makeMove`

```
template <bool EXACT = false>
void makeMove(const Move move);
```

Метода `makeMove` је шаблонска метода са параметром шаблона `EXACT` типа `bool`, чија је подразумевана вредност `false`. Када је овај параметар `true`, додатно се проверава да ли *en passant* потез легалан. У раду се користи искључиво подразумевана вредност параметра `EXACT`. Параметар `move` типа `Move` представља потез који треба да се одигра. Прослеђени потез мора бити легалан, у супротном је понашање недефинисано. О овом механизму је више речено у потпоглављу 3.2.2 Класа `Move`.

Пример методе за спровођење правила шаха над тренутном позицијом је метода која проверава да ли је дошло до краја партије – `isGameOver`. Декларација методе `isGameOver` је приказана у код листингу 3-3 [19].

Код листинг 3-3 Декларација методе `isGameOver`

```
std::pair<GameResultReason, GameResult> isGameOver() const;
```

Повратна вредност методе је пар `std::pair<GameResultReason, GameResult>`. Први члан је вредност набројивог типа `GameResultReason` и представља разлог исхода партије, а други члан је вредност набројивог типа `GameResult` и представља исход партије из перспективе играча који је на потезу. Дефиниција набројивог типа `GameResultReason` је дата у код листингу 3-4 [19].

Код листинг 3-4 Дефиниција набројивог типа `GameResultReason`

```
enum class GameResultReason {
    CHECKMATE,
    STALEMATE,
    INSUFFICIENT_MATERIAL,
    FIFTY_MOVE_RULE,
    THREEFOLD_REPETITION,
    NONE
};
```

Разлог исхода партије може бити:

- шах-мат – `CHECKMATE`
- реми услед пата – `STALEMATE`
- реми услед недовољно материјала – `INSUFFICIENT_MATERIAL`
- реми услед правила 50 потеза – `FIFTY_MOVE_RULE`
- реми услед правила понављања позиције три пута – `THREEFOLD_REPETITION`
- партија још увек траје, односно постоји још легалних потеза у тренутној позицији – `NONE`.

Дефиниција набројивог типа `GameResult` је дата у код листингу 3-5 [19].

Код листинг 3-5 Дефиниција набројивог типа `GameResult`

```
enum class GameResult { WIN, LOSE, DRAW, NONE };
```

Исход партије из перспективе играча који је на потезу може бити:

- играч је победио – `WIN`
- играч је изгубио – `LOSE`
- реми – `DRAW`
- партија још увек траје, односно постоји још легалних потеза у тренутној позицији – `NONE`.

Метода `getFen` као повратну вредност враћа тренутни *FEN* стринг. Декларација методе `getFen` је дата у код листингу 3-6 [19].

Код листинг 3-6 Декларација методе `getFen`

```
std::string getFen(bool moveCounters = true) const;
```

Позивом методе са подразумеваном вредношћу параметра се као повратна вредност враћа *FEN* стринг са бројем полупотега и бројем целих потега. Уколико је вредност параметра `false`, они се не приказују у повратној вредности.

Декларација методе `at` дата у код листингу 3-7 [19].

Код листинг 3-7 Декларација методе `at`

```
template <typename T = Piece>  
T at(Square sq) const;
```

`at` је шаблонска метода чији тип повратне вредности одређује параметар шаблона `T`. У раду се користи подразумевани тип `Piece`. `Piece` је класа која представља конкретну фигуру, одређену и бојом и типом фигуре. Параметар методе је типа `Square`. `Square` је класа која представља одређено поље табле. Метода дакле, за прослеђено поље као повратну вредност враћа фигуру на том пољу.

3.2.1.1 Класе `Piece`, `PieceType`, `Color` и `Square`

У раду се инстанцирање класе `Piece` врши конструктором без параметара. Класа `Piece` користи набројиви тип `underlying` чија је дефиниција приказана у код листингу 3-8 [19].

Код листинг 3-8 Дефиниција набројивог типа `Piece::underlying`

```
enum class underlying : std::uint8_t {  
    WHITEPAWN,  
    WHITEKNIGHT,  
    WHITEBISHOP,  
    WHITEROOK,  
    WHITEQUEEN,  
    WHITEKING,  
    BLACKPAWN,  
    BLACKKNIGHT,  
    BLACKBISHOP,  
    BLACKROOK,  
    BLACKQUEEN,  
    BLACKKING,  
    NONE  
};
```

Набројиви тип `underlying` дефинише скуп набројивих вредности које представљају све могуће фигуре, од белог пешака – `WHITEPAWN` до црног краља – `BLACKKING`, као и одсуство фигуре на датом пољу – `NONE`.

Класа `Piece` садржи и статичка поља истих имена као набројиве вредности типа `underlying`. Сваком од ових поља додељена је одговарајућа набројива вредност, као што је приказано у код листингу 3-9 [19].

Код листинг 3-9 Статичка поља класе `Piece` типа `Piece::underlying`

```
static constexpr underlying NONE           = underlying::NONE;  
static constexpr underlying WHITEPAWN     = underlying::WHITEPAWN;  
static constexpr underlying WHITEKNIGHT   = underlying::WHITEKNIGHT;  
static constexpr underlying WHITEBISHOP   = underlying::WHITEBISHOP;  
static constexpr underlying WHITEROOK     = underlying::WHITEROOK;  
static constexpr underlying WHITEQUEEN    = underlying::WHITEQUEEN;  
static constexpr underlying WHITEKING     = underlying::WHITEKING;  
static constexpr underlying BLACKPAWN     = underlying::BLACKPAWN;  
static constexpr underlying BLACKKNIGHT   = underlying::BLACKKNIGHT;  
static constexpr underlying BLACKBISHOP   = underlying::BLACKBISHOP;
```

```
static constexpr underlying BLACKROOK = underlying::BLACKROOK;
static constexpr underlying BLACKQUEEN = underlying::BLACKQUEEN;
static constexpr underlying BLACKKING = underlying::BLACKKING;
```

На овај начин се истој набројивој вредности може приступати на два начина: преко набројивог типа, на пример `Piece::underlying::WHITEPAWN`, или преко статичког поља класе, тј. `Piece::WHITEPAWN`.

Додатно, користе се и методе `type` и `color`, помоћу којих је могуће добити информацију о типу шаховске фигуре и боји шаховске фигуре. Декларације ових метода су дате у код листингу 3-10 [19].

Код листинг 3-10 Декларације метода `Color::color` и `PieceType::type`

```
constexpr Color color() const;
constexpr PieceType type() const;
```

Класа `Color` представља боју. Користи набројиви тип `underlying` чија је дефиниција приказана у код листингу 3-11 [19].

Код листинг 3-11 Дефиниција набројивог типа `Color::underlying`

```
enum class underlying : std::int8_t {
    WHITE = 0,
    BLACK = 1,
    NONE = -1
};
```

Набројиви тип `underlying` дефинише скуп набројивих вредности које представљају боје: белу боју – `WHITE`, црну боју – `BLACK`, и додатну вредност – `NONE`. Класа `Color` садржи и статичка поља истих имена као набројиве вредности типа `underlying`. Ово је идентичан механизам као и у класи `Piece`, који омогућава приступ истој набројивој вредности преко набројивог типа или преко статичког поља класе.

Класа `PieceType` је класа која представља тип шаховске фигуре, независно од њене боје. У раду се користи конструктор без параметара. Класа `PieceType` користи набројиви тип `underlying` чија је дефиниција приказана у код листингу 3-12 [19].

Код листинг 3-12 Дефиниција набројивог типа `PieceType::underlying`

```
enum class underlying : std::uint8_t {
    PAWN,
    KNIGHT,
    BISHOP,
    ROOK,
    QUEEN,
    KING,
    NONE
};
```

Набројиви тип `underlying` дефинише скуп набројивих вредности које представљају све типове фигура од пешака – `PAWN`, до краља – `KING`, као и додатна вредност – `NONE`. Класа `PieceType` садржи и статичка поља истих имена као набројиве вредности типа `underlying`. Ово је идентичан механизам као и у класама `Piece` и `Color`.

У раду се инстанцирање класе `Square` врши конструктором са целобројним параметром из опсега [0, 64]. Поље на физичкој табли `a1` одговара броју 0, а поље `h8` одговара броју 63. Број 64 не представља валидно поље шаховске табле. Конструктор је приказан у код листингу 3-13 [19].

Код листинг 3-13 Декларација конструктора са параметром класе `Square`

```
constexpr Square(int sq);
```

Класа `Square` користи набројиви тип `underlying` чија је дефиниција приказана у код листингу 3-14 [19].

```
enum class underlying : uint8_t {
    SQ_A1, SQ_B1, SQ_C1, SQ_D1, SQ_E1, SQ_F1, SQ_G1, SQ_H1,
    SQ_A2, SQ_B2, SQ_C2, SQ_D2, SQ_E2, SQ_F2, SQ_G2, SQ_H2,
    SQ_A3, SQ_B3, SQ_C3, SQ_D3, SQ_E3, SQ_F3, SQ_G3, SQ_H3,
    SQ_A4, SQ_B4, SQ_C4, SQ_D4, SQ_E4, SQ_F4, SQ_G4, SQ_H4,
    SQ_A5, SQ_B5, SQ_C5, SQ_D5, SQ_E5, SQ_F5, SQ_G5, SQ_H5,
    SQ_A6, SQ_B6, SQ_C6, SQ_D6, SQ_E6, SQ_F6, SQ_G6, SQ_H6,
    SQ_A7, SQ_B7, SQ_C7, SQ_D7, SQ_E7, SQ_F7, SQ_G7, SQ_H7,
    SQ_A8, SQ_B8, SQ_C8, SQ_D8, SQ_E8, SQ_F8, SQ_G8, SQ_H8,
    NO_SQ
};
```

Набројиви тип `underlying` дефинише скуп набројивих вредности које представљају сва поља табле, као и додатну вредност – `NO_SQ`. Поље на физичкој табли *a1* одговара набројивој вредности `SQ_A1`, а поље *h8* одговара набројивој вредности `SQ_H8`. `NO_SQ` одговара броју 64 и не представља валидно поље шаховске табле. Сличан механизам као и у класама `Piece`, `Color` и `PieceType` омогућава приступ набројивим вредностима преко типа `Square::underlying` или преко статичких чланова класе.

3.2.2 Класа `Move`

Класа `Move` представља потез. Потез је одређен пољем са којег је фигура подигнута и поље на које је фигура спуштена. Овакво представљање потеза је слично као и у универзалном шаховском интерфејсу (енгл. *Universal Chess Interface – UCI*) који је детаљно објашњен у потпоглављу 3.3 *Stockfish*. Једини изузетак је рокада. Она се записује као потез у коме је почетно поље поље краља (*e1* или *e8*), а крајње поље поље топа (*a1/h1* за играча са белим фигурама или *a8/h8* за играча са црним фигурама).

Класа има статичка поља која представљају типове потеза. Статичка поља су приказана у код листингу 3-15 [19].

Код листинг 3-15 Статичка поља класе `Move`

```
static constexpr std::uint16_t NO_MOVE      = 0;
static constexpr std::uint16_t NULL_MOVE   = 65;
static constexpr std::uint16_t NORMAL      = 0;
static constexpr std::uint16_t PROMOTION   = 1 << 14;
static constexpr std::uint16_t ENPASSANT   = 2 << 14;
static constexpr std::uint16_t CASTLING    = 3 << 14;
```

Ова поља се користе за пролазак кроз листу свих дозвољених потеза, на пример проналазак дозвољених промоција пешака – `PROMOTION`, *en passant* заробљавања – `ENPASSANT` и рокада – `CASTLING`.

Неке од помоћних метода које класа `Move` нуди су `typeOf`, `from`, `to` и `promotionType`. Декларације ових метода су дате у код листингу 3-16 [19].

Код листинг 3-16 Декларације метода класе `Move`

```
Square from() const;
Square to() const;
uint16_t typeOf() const;
PieceType promotionType() const;
```

Метода `typeOf` као повратну вредност враћа тип потеза дефинисан једним од статичких поља у код листингу. Метода `from` враћа поље са којег је потез започет, а метода `to` враћа поље на којем је потез завршен. Метода `promotionType` као повратну вредност враћа тип фигуре у коју је пешак промовисан.

Додатно, постоји класа `uci`, чија метода `moveToUci` се користи како би се потез класе `Move` претворио у стринг који представља *UCI* запис потеза. Декларација методе је приказана у код листингу 3-17 [19].

```
std::string moveToUci(const Move& move, bool chess960 = false);
```

Као и за конструктор класе `Board`, пошто се овом раду неће разматрати *Fisher Random* шах, за други параметар користи подразумевана вредност.

3.2.3 Структура `Movelist` и класа `movegen`

Структура `Movelist` представља контејнер потеза са методама за извршавање операција сличним онима које би се извршавале над листама – итерација кроз контејнер, провера да ли је контејнер празан, добијање информације о његовој величини и слично.

Класа `movegen` се користи за генерисање легалних потеза у одређеној позицији. Метода која се користи за ту сврху је `legalmoves`. Декларација ове методе је дата у код листингу 3-18 [19], [20].

Код листинг 3-18 Декларација методе `movegen::legalmoves`

```
template <MoveGenType mt = MoveGenType::ALL>
void static legalmoves(
    Movelist& movelist,
    const Board& board,
    int pieces = PieceGenType::PAWN |
                PieceGenType::KNIGHT |
                PieceGenType::BISHOP |
                PieceGenType::ROOK |
                PieceGenType::QUEEN |
                PieceGenType::KING
);
```

Метода `legalmoves` је шаблонска метода са параметром шаблона `mt` типа `MoveGenType`. `MoveGenType` је набројиви тип приказан у код листингу 3-19 [19].

Код листинг 3-19 Дефиниција набројивог типа `MoveGenType`

```
enum class MoveGenType : std::uint8_t { ALL, CAPTURE, QUIET };
```

Набројиви тип `MoveGenType` дефинише скуп врста потеза који се могу одиграти у позицији; те врсте су: било који – `ALL`, заробљавања – `CAPTURE` и потези који нису заробљавања – `QUIET`.

Подразумевана вредност параметра шаблона `mt` је вредност `ALL`. Метода `legalmoves` има три параметра. Параметар `movelist` је контејнер потеза у који ће се поставити сви излистани потези. Параметар `board` је тренутно стање табле за кога се генеришу могући легални потези, а параметар `pieces` указује на за које фигуре се генерише листа потеза.

`PieceGenType` је набројиви тип података дефинисан у код листингу 3-20 [19].

Код листинг 3-20 Дефиниција набројивог типа `PieceGenType`

```
enum PieceGenType {
    PAWN    = 1,
    KNIGHT  = 2,
    BISHOP  = 4,
    ROOK    = 8,
    QUEEN   = 16,
    KING    = 32,
};
```

Целобројне вредности заступају фигуре за које се генерише листа потеза. Помоћу битског ИЛИ се могу комбиновати фигуре за које су потези од интереса. Подразумевана вредност параметра `pieces` указује да се потези излиставају за све фигуре.

3.3 Stockfish

Stockfish је бесплатан шаховски програм отвореног кода. Користи се да анализира шаховске позиције и израчуна оптималан потез у датој позицији. *Stockfish* користи *UCI* протокол и не поседује графички кориснички интерфејс, који приказује шаховску таблу и олакшава унос потеза [21].

У овом раду је развијен кориснички интерфејс, по узору на типичне корисничке интерфејсе који се бесплатно могу преузети онлајн. Помоћу њега ће апликација интераговати са *Stockfish* извршним програмом, тако што ће покретати програм, задавати команде преко његовог стандардног улаза (*stdin*) и чекати *Stockfish* излаз, који се појављује на његовом стандардном излазу (*stdout*).

3.3.1 UCI протокол

UCI, односно универзални шаховски протокол, је текстуални протокол који се користи за комуникацију са шаховским програмом. Ово је препоручени начин комуникације за типичне графичке корисничке интерфејсе или шаховске алате [21]. Овде су излистане стандардне команде које се користе за интеракцију са *Stockfish* програмом.

uci командом се говори програму да пређе у *UCI* режим – користи *UCI*. Команда се шаље једном, одмах након што се *Stockfish* програм покрене. *Stockfish* програм ће послати *uciok* чиме означава да је прихватио *UCI* режим. Уколико се не пошаље *uciok* након одређеног времена, *Stockfish* програм би требало да се терминира [21].

isready се користи за синхронизацију *Stockfish* програма и апликације. Користи се како би се сачекало да *Stockfish* програм заврши са обрадом претходних дуготрајних задатака или да се провери да ли је *Stockfish* идаље активан. На команду *Stockfish* одговара са *readyok* [21].

position се користи за поставку тренутне шаховске позиције у *Stockfish* програму. Ово је могуће урадити на два начина:

- *position fen <fenstring>* – постављање позиције на основу њеног описа *FEN* стрингом *fenstring*
- *position startpos* – постављање позиције почетну, *startpos*.

Додатно је могуће коришћење *moves* команде како би се даље наводили одиграни потези, на пример, *position startpos moves e2e4*. Потези се наводе као комбинација почетног и одредишног поља фигуре којом се потез врши, на пример, *e2e4*. У случају рокаде, фигура која се прати је краљ, па је тако на пример мала рокада играча са белим фигурама *e1g1*. У случају промоције пешака, потез се бележи са додатним малим словом абецеде које означава у шта се промоција врши. Краљица се обележава са *q*, топ са *r*, ловац са *b*, а коњ са *n*. На пример, промоција у краљицу на пољу *g8* је *g7g8q* [21] [22].

go је команда за почетак рачунања на позицији постављеној од стране команде *position*. Постоји много параметара који служе да ограниче претрагу оптималног потеза, као на пример *movetime <x>*, који ограничава претрагу на приближно *x ms*. *stop* команда служи за прекидање рачунања што је пре могуће [21].

3.4 C библиотека за рад са Arduino USART-ом

За рад са *USART0* модулом се користи *C* библиотека која се састоји из *usart.c* и *usart.h* датотека [5], [23]. За реализацију овог рада су коришћене две функције – *usartInit* и *usartPutChar*.

Декларација функције *usartInit* је дата у код листингу 3-21 [23].

```
void usartInit(uint32_t baud);
```

Функција иницијализује регистре `USART0` модула, а као параметар прима жељену брзину серијске комуникације изражену у bit/s.

Декларација функције `usartPutChar` је дата у код листингу 3-22 [23].

```
void usartPutChar(int8_t c);
```

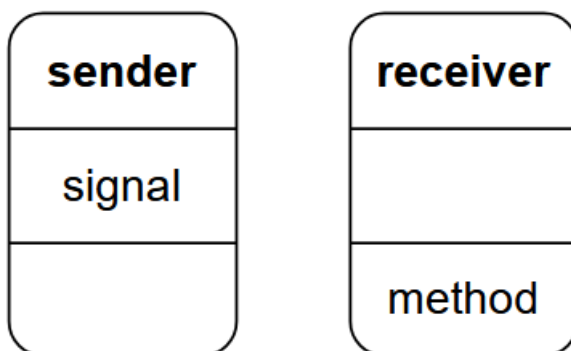
Функција преко серијског порта шаље карактер који јој се прослеђује као параметар.

3.5 Qt радни оквир

За развој графичког корисничког интерфејса користиће се Qt5 C++ библиотека. Њене класе, поред поља и метода, могу садршати и сигнале и слотове који служе за комуникацију између објеката графичког корисничког интерфејса. Комуникација између објеката графичког корисничког интерфејса се заснива на догађајима; када се деси неки догађај везан за објекат, објекат ће емитовати неки од својих сигнала. Објекат може емитовати сигнал из било које друге методе или слота те класе. Сигнал се емитује употребом кључне речи `emit` испред навођења сигнала са одговарајућим параметрима [1].

Слот је метода која представља одговор неког објекта на емитовани сигнал, под условом да су емитовани сигнал и слот повезани [1]. Сигнал и слот морају имати параметре истог типа и наведене истим редоследом. Слот сме имати мање параметара него сигнал и у том случају само игнорише додатне параметре сигнала [24]. Повезани сигнал и слот могу припадати истом објекту.

Нека један објекат, под називом `sender`, има сигнал `signal`, а други објекат, под називом `receiver`, има слот `method`, као што је приказано на слици 3-2.



Слика 3-2 Објекти `sender` и `receiver`

Они се повезују методом `connect`, чија је декларација приказана у код листингу 3-23 [24].

```
QMetaObject::Connection QObject::connect(  
    const QObject *sender,  
    const char *signal,  
    const QObject *receiver,  
    const char *method,  
    Qt::ConnectionType type = Qt::AutoConnection  
);
```

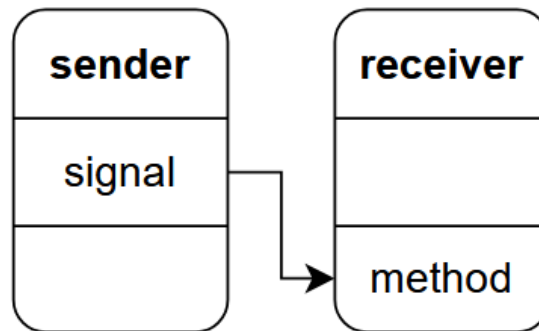
Методом `connect` се успоставља веза типа `type` од сигнала `signal` у `sender` објекту до слота `method` у `receiver` објекту. Повратна вредност је хендл на везу и може се

користити касније да би се веза прекинула [24]. Пример позива `connect` је приказан у код листингу 3-24.

Код листинг 3-24 Пример позива `connect`

```
connect(  
    sender,  
    SIGNAL(signal(parameter types)),  
    receiver,  
    SLOT(method(parameter types))  
);
```

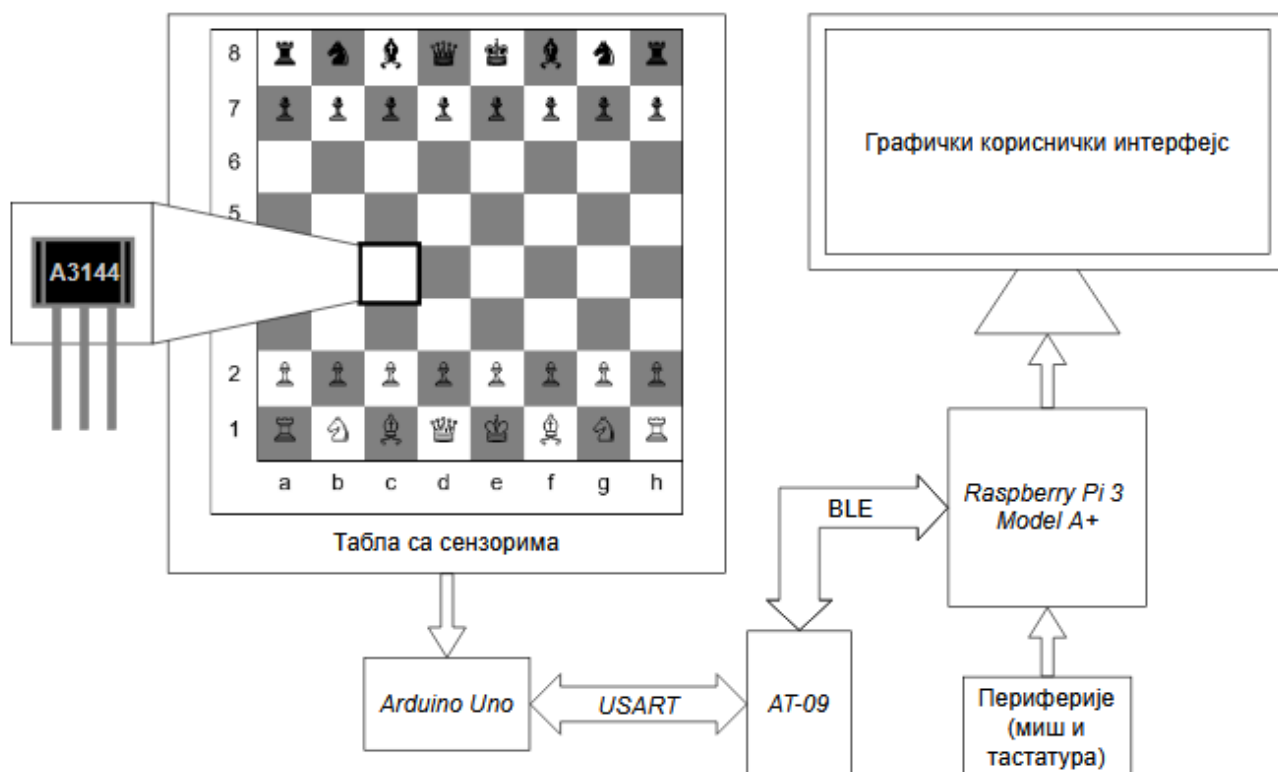
Користе се `SIGNAL()` и `SLOT()` макрои приликом специфицирања сигнала и слота у вези. Параметри сигнала и слота наводе само листу типова параметара, без имена [24]. Слика 3-3 представља везу између објеката `sender` и `receiver` након позива методе `connect`.



Слика 3-3 Објекти након позива методе `connect`

4. РЕАЛИЗАЦИЈА РАДА

Архитектура система је приказана на слици 4-1.



Слика 4-1 Архитектура система

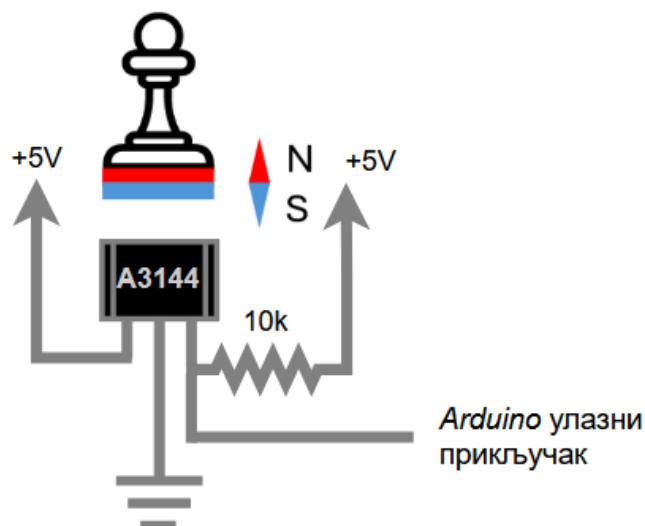
Arduino микроконтролер добија очитавања са сензора преко својих улазно-излазних прикључака. Ти подаци се форматирају и шаљу преко серијске комуникације *BLE* модулу *AT-09*. Са друге стране, када се успостави веза *AT-09* и *Raspberry Pi*-ја, *Raspberry Pi* добија тренутно стање табле и на основу апликације која се на њему врши, ажурира графички кориснички интерфејс. Корисник са системом интерагује преко фигура, тастатуре и миша.

4.1 Хардверска реализација

4.1.1 Табла са сензорима

Hall сензор је униполарни и у присуству јужног пола магнета спушта излазни напон на ниску вредност. Ова чињеница се користи да би се детектовала присутност фигура на пољима. Испод сваког поља табле се налази један дигитални *Hall* сензор. На дну сваке фигуре је фиксиран стални магнет са јужним пољем окренутим ка пољу табле. Због тога, излаз *Hall* сензора придруженог пољу на коме је фигура даје ниску вредност, док *Hall* сензор придружен пољу на коме нема фигуре треба да даје високу вредност.

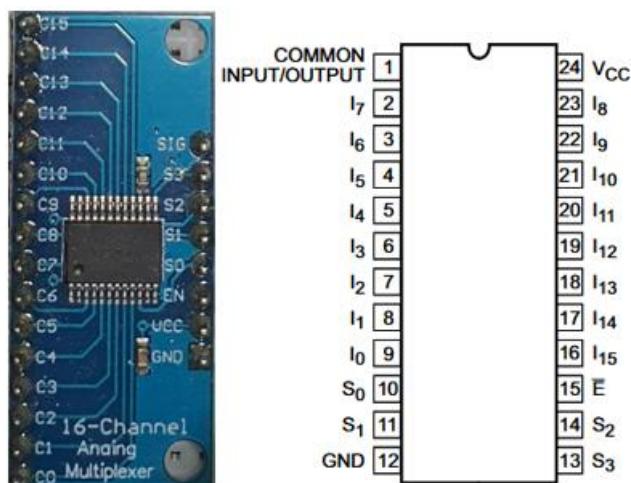
Излазни прикључак *Hall* сензора је отворени колектор, стога се при повезивању са улазно-излазним прикључком микроконтролера мора увести *pull-up* отпорник ка напајању. *Hall* сензор придружен пољу на коме нема фигуре тиме даје високу вредност. Шема повезивања једног *Hall* сензора је приказана на слици 4-2.



Слика 4-2 Шема повезивања једног Hall сензора

Укупан број излаза *Hall* сензора је много већи од укупног броја *Arduino* улазно-излазних прикључака. Са сваког поља табле полази један прикључак, па је укупно 64 прикључка потребно спојити са *Arduino* развојним системом. Директна веза не може да се успостави, с обзиром да плочица има свега 20 доступних улазно-излазних прикључака. Као решење се уводи мултиплексирање излаза сензора, за шта се користе четири модула који ће служити као 16 канални мултиплексери.

На модулу се налази чип *CD74HC4067* компаније *Texas Instruments*. У питању је аналогни мултиплексер/демултиплексер. Они су бидирекциони прекидачи и стога било који аналогни улаз може бити излаз и обрнуто. Изглед модула и распоред прикључака на чипу модула је приказан на слици 4-3 [25].



Слика 4-3 Изглед модула (лево) и распоред прикључака на чипу *CD74HC4067* тог модула (десно)

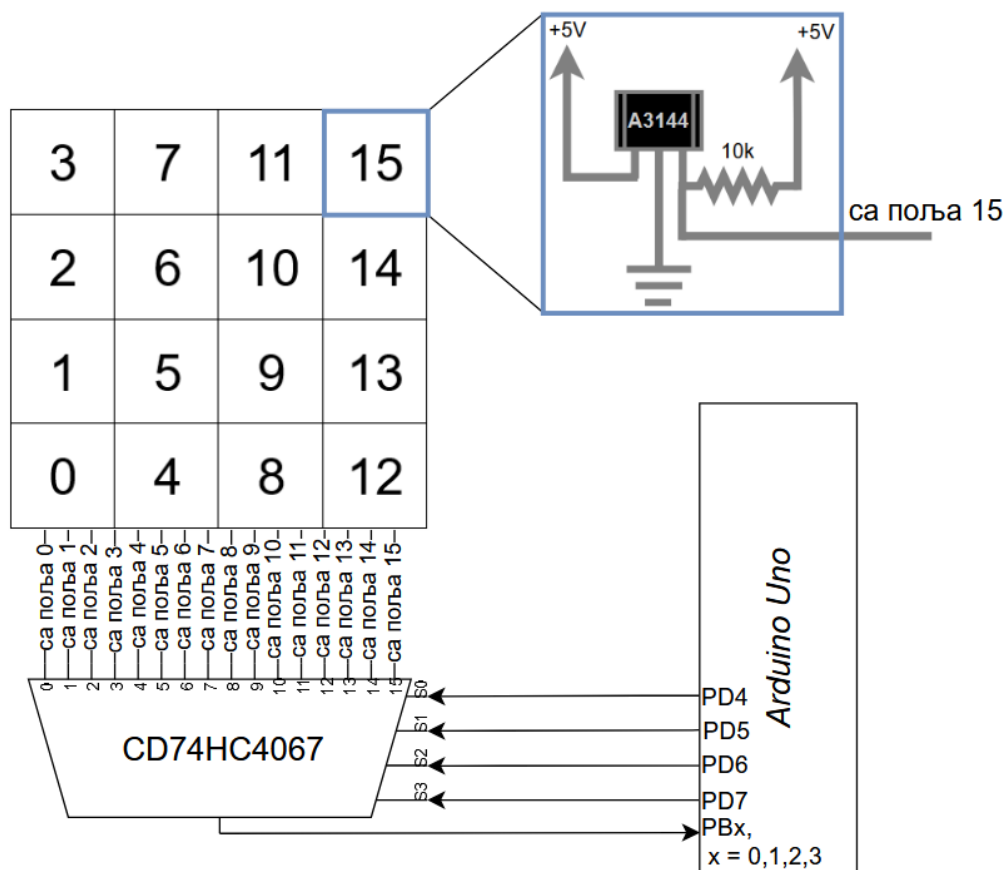
За овај систем ће *CD74HC4067* представљати мултиплексер. Прикључци *I₀-I₁₅* имају улогу информационих улаза. Прикључак *COMMON INPUT/OUTPUT* (на самом модулу означен са *SIG*) представља излаз – садржај једног од 16 информационих улаза. Прикључци *S₀-S₃* представљају селекционе улазе. На основу њих се бира који од информационих улаза ће бити пропуштен на *COMMON INPUT/OUTPUT* излаз. *V_{CC}* и *GND* су напајање и маса, опсег напајања који се користи је од 2 V до 6 V. У раду се користи 5 V. *E* прикључак представља сигнал дозволе. Истинитосна таблица чипа је приказана табелом 4-1 [25].

Табела 4-1 Истинитосна таблица чипа CD74HC4067 [25]

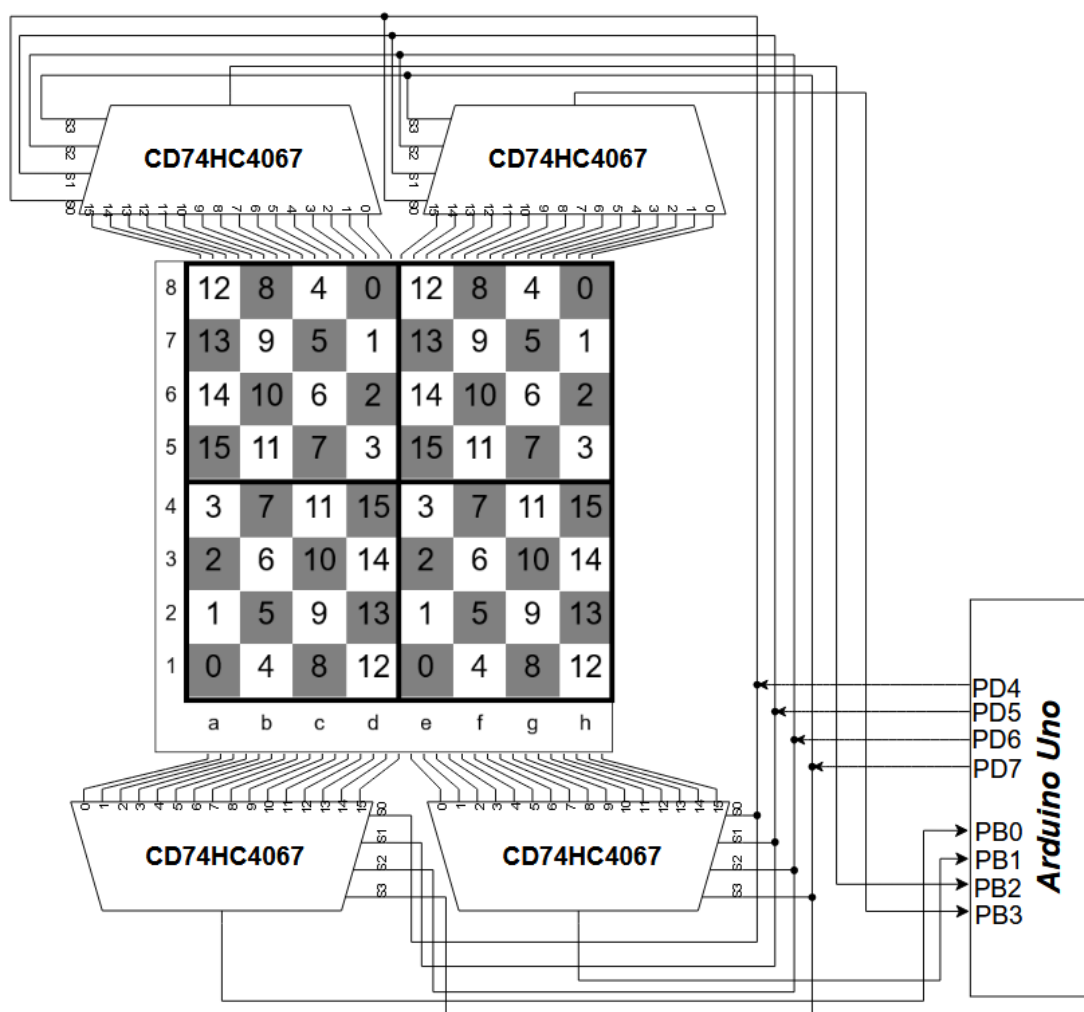
S_0	S_1	S_2	S_3	$\bar{E}$	Изабрани канал I_x који се прослеђује на COMMON INPUT/OUTPUT
X	X	X	X	1	Ниједан
0	0	0	0	0	0
1	0	0	0	0	1
0	1	0	0	0	2
1	1	0	0	0	3
0	0	1	0	0	4
1	0	1	0	0	5
0	1	1	0	0	6
1	1	1	0	0	7
0	0	0	1	0	8
1	0	0	1	0	9
0	1	0	1	0	10
1	1	0	1	0	11
0	0	1	1	0	12
1	0	1	1	0	13
0	1	1	1	0	14
1	1	1	1	0	15

Увођењем мултиплексера 16 на 1 табла се дели на 4 идентична сегмента, свако поље сегмента се индексира бројевима 0-15, који одговарају индексима информационих улаза мултиплексера са којим су повезани. Селекциони сигнали су међусобно повезани. S_3 канали сваког мултиплексера су међусобно инцидентни и исто важи и за S_2 , S_1 и S_0 . Ово није ограничавајуће јер ће *Arduino* пролазити кроз сегменте секвенцијално, један по један и за сваки гледати излазни сигнал мултиплексера *SIG* који му је у том тренутку од интереса.

Један сегмент табле је приказан на слици 4-4. Са сваког поља се излазни сигнал сензора повезује са одговарајућим информационим улазом. Селекциони сигнали мултиплексера су повезани са прикључцима 4-7 порта *Port D*. Ови прикључци порта *Port D* су стога излазни за микроконтролер. Излазни сигнали мултиплексера се доводе на прикључке 0-3 порта *Port B*. Ови прикључци порта *Port B* су стога улазни за микроконтролер. Цела табла, распоред мултиплексера и начин повезивања са *Arduino* развојним системом су приказани на слици 4-5.



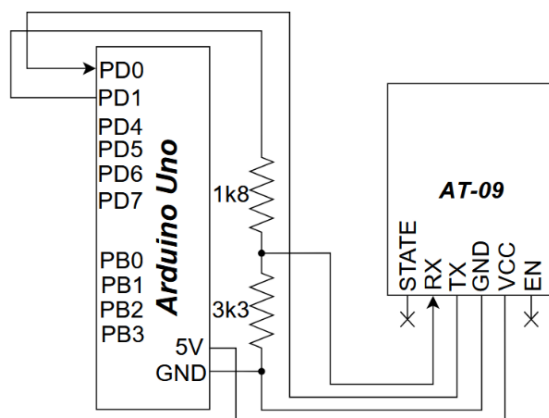
Слика 4-4 Повезивање једног сегмента са мултиплтксером и Arduino-ом



Слика 4-5 Распоред мултиплексера и повезивање са Arduino-ом

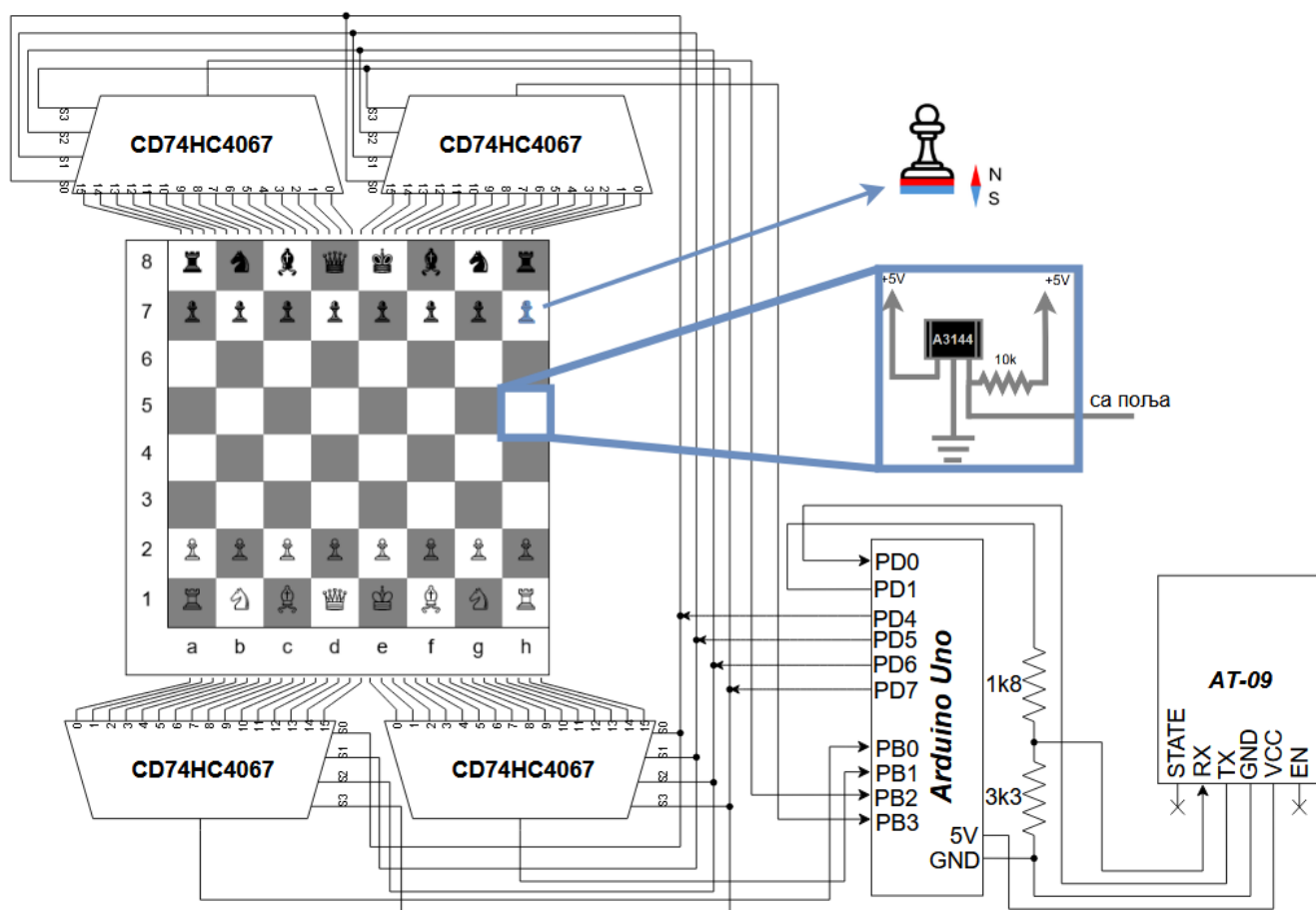
4.1.2 Повезивање *Arduino* развојног система и модула *AT-09*

Arduino и *AT-09* комуницирају путем *USART* серијске комуникације. Стање шаховске табле се шаље модулу *AT-09*, да би он могао да обезбеди податке за *Raspberry Pi*. При повезивању ова два уређаја потребно је узети у обзир разлику у логичким нивоима. *Arduino* користи логички ниво од 5 V, док је за улаз *RX* модула *AT-09* потребан логички ниво од 3.3 V. Због тога се између прикључка *PD1* порта *Port D* *Arduino*-а, који представља излаз за пренос података у серијској комуникацији (на штампаној плочици поред прикључка стоји ознака *TX*), и прикључка *RX* модула *AT-09* користи напонски разделник за снижавање напона на 3.3 V. Шема повезивања *Arduino* развојног система и модула *AT-09* је приказана на слици 4-6.



Слика 4-6 Шема повезивања *Arduino* развојног система и модула *AT-09*

Шема комплетне шаховске табле је приказана на слици 4-7.



Слика 4-7 Шема комплетне шаховске табле

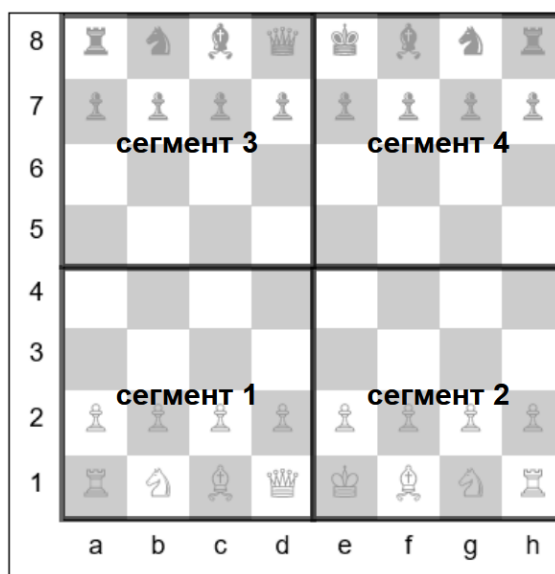
4.2 Реализација програмске подршке

4.2.1 *Arduino* програм за обилазак поља табле

На слици 4-5 је приказан распоред мултиплексера и начин на који су њихови *SIG* канали повезани на *Arduino* прикључке *PB0-PB3* порта *Port B*. Мултиплексерима је табла подељена на 4 сегмента. Сегменти се за потребе развоја програма за обилазак поља табле индексирају на следећи начин:

- први обухвата колоне *a-d* и редове 1-4, а *SIG* канал мултиплексера повезаног са овим пољима повезан је на прикључак *PB0*
- други обухвата колоне *e-h* и редове 1-4, а *SIG* канал мултиплексера повезаног са овим пољима повезан је на прикључак *PB1*
- трећи обухвата колоне *a-d* и редове 5-8, а *SIG* канал мултиплексера повезаног са овим пољима повезан је на прикључак *PB3*
- четврти обухвата колоне *e-h* и редове 5-8, а *SIG* канал мултиплексера повезаног са овим пољима повезан је на прикључак *PB4*.

Слика 4-8 приказује ову сегментацију.



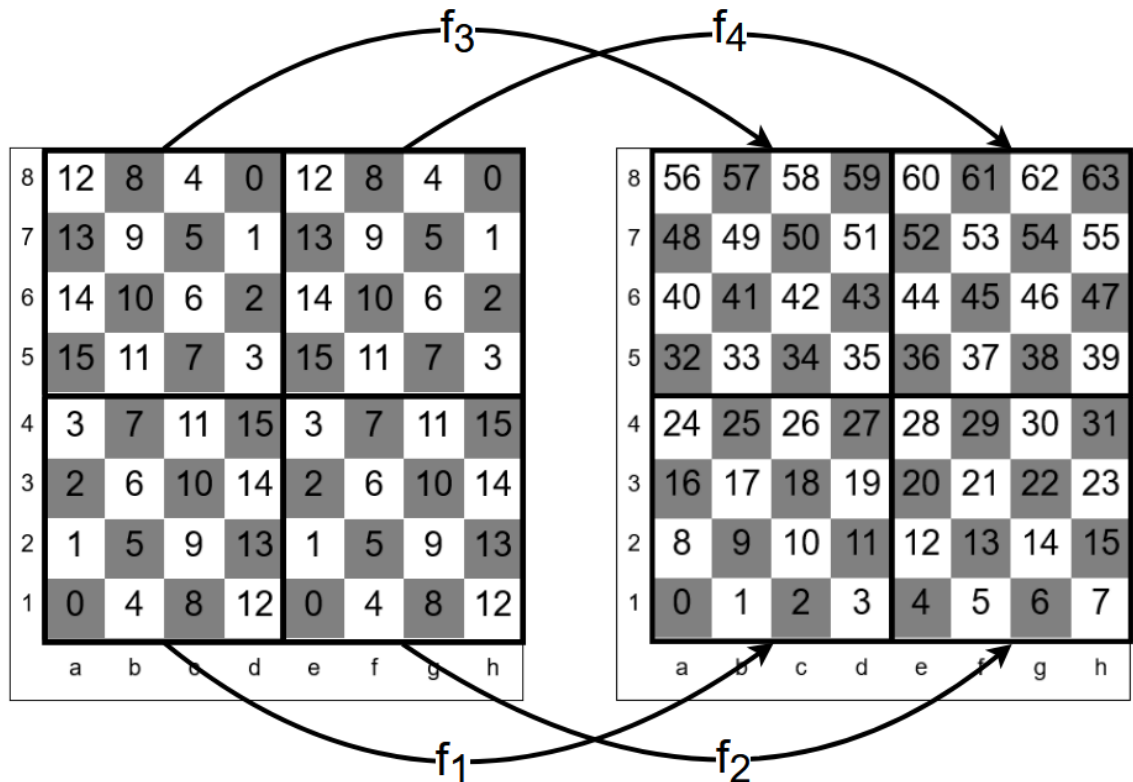
Слика 4-8 Сегментација шаховске табле

Кроз таблу се пролази редом по сегментима, почевши од првог. Поља свих сегмента се адресирају помоћу прикључака *PD3-PD7* порта *Port D*. *PD7* је повезан са свим S_3 каналима мултиплексера, *PD6* са свим S_2 каналима, *PD5* са свим S_1 каналима и *PD4* са свим S_0 каналима.

Унутар програма, табла се представља низом *new_state* од 64 неозначена цела броја. Уколико се на пољу налази фигура, излаз *Hall* сензора ће дати ниску логичку вредност. Ако је на реду за обилазак сегмента са тим пољем, када се адресира тај информациони улаз, одговарајући прикључак порта *Port B* ће бити на ниској логичкој вредности, самим тим ће се нула уписати као вредност елемента низа који одговара том пољу. Ефективно, када је поље окупирано фигуром, одговарајући елемент у низу *new_state* је једнак 0, а у супротном је једнак 1. Елемент *new_state[0]* одговара пољу *a1*, а елемент *new_state[63]* пољу *h8*.

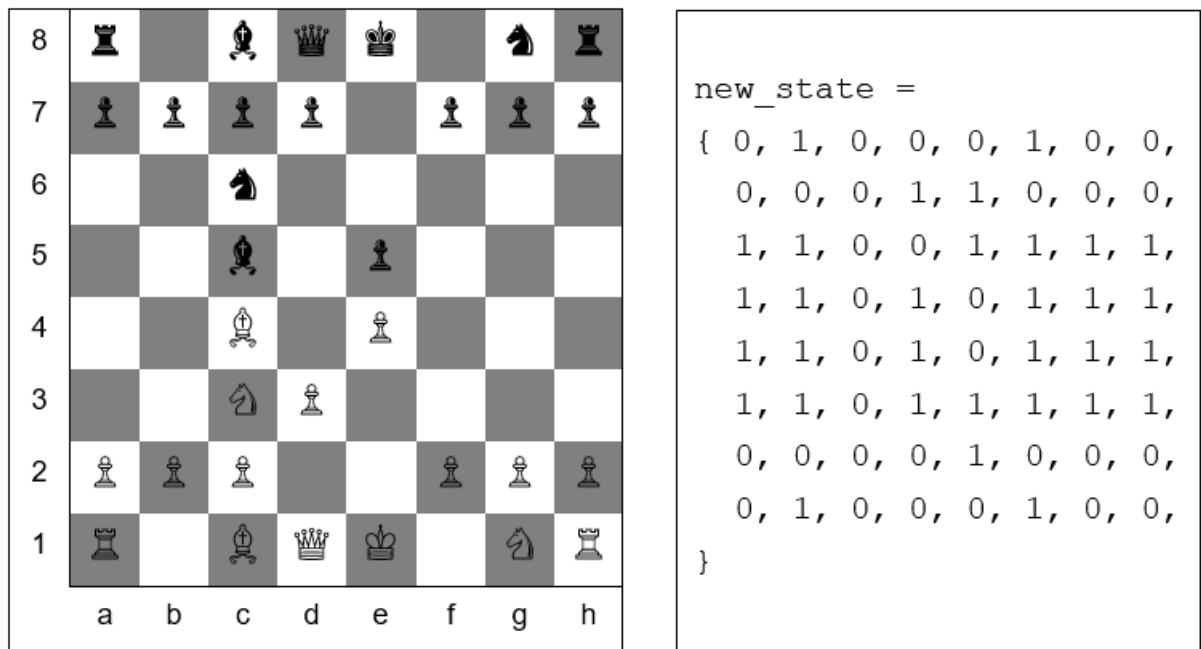
Остало је још довести у везу индексирање поља табле по сегментима, онако како је мултиплексери деле, индексима 0-15 и одговарајуће индексе за приступ елементима низа *new_state*, где се сваком пољу додељује индекс 0-63. Ако се табла индексирана на начин који одговара индексима *new_state* низа подели на сегменте као на слици 4-8, веза међу индексима се може поделити на 4 пресликавања. Свако пресликавање

пресликава индексе 0-15 у индексе одговарајућих сегмената низа `new_state`. Слика 4-9 илуструје ова пресликавања.



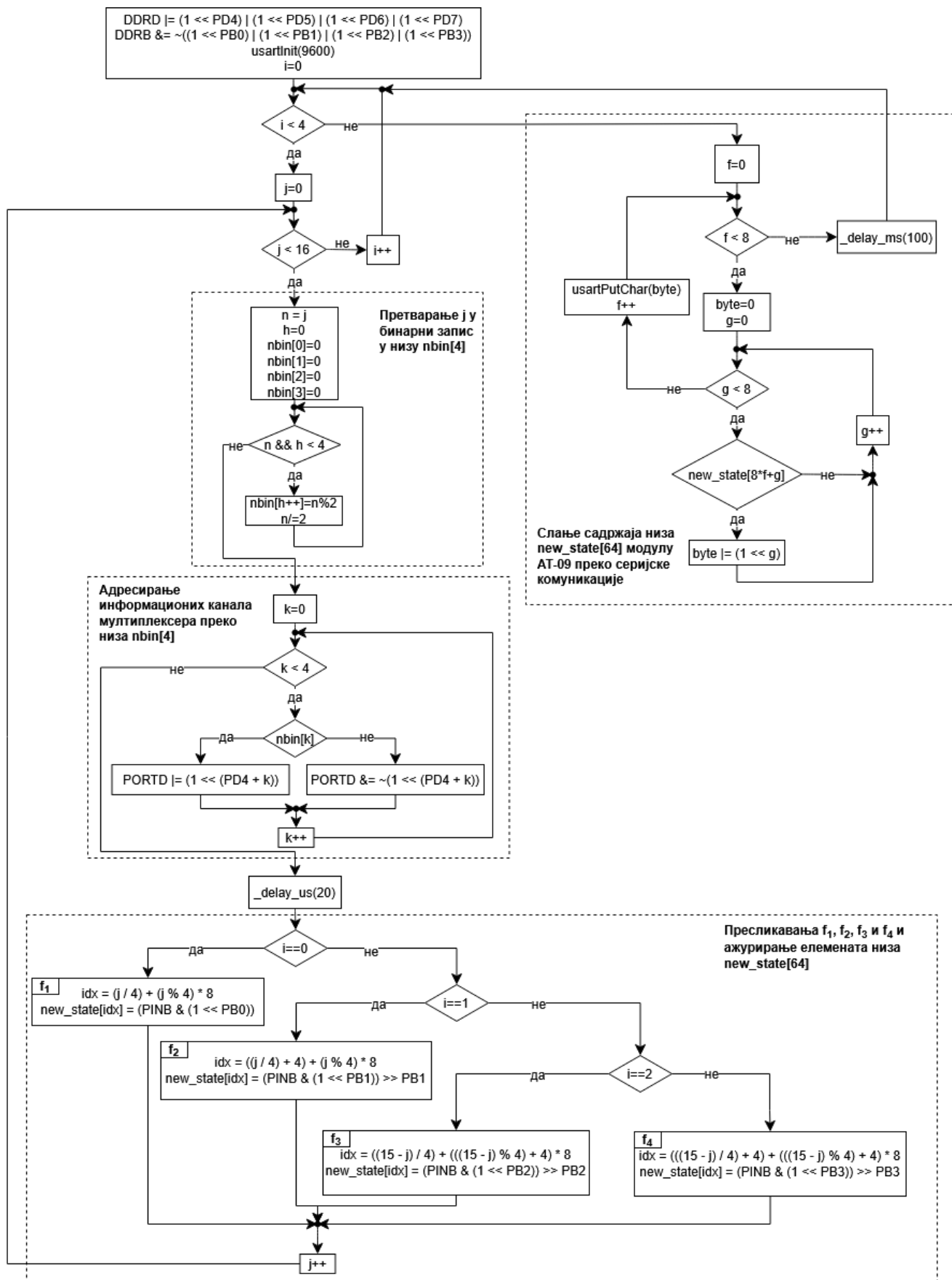
Слика 4-9 Пресликавање индексирања спроведеног мултиплексерима у индексирање елемената низа `new_state`

Слика 4-10 са десне стране приказује пример садржаја `new_state` низа након што се достигне позиција на табли приказана лево.



Слика 4-10 Садржај низа `new_state` (десно) за приказану позицију (лево)

Након обилазке табле, користи се функција `usartPutChar` да пошаље, врсту по врсту, садржај низа `new_state` на `RX` прикључак `AT-09` модула. Алгоритам рада програма на *Arduino* развојном систему је приказан на слици 4-11.



Слика 4-11 Алгоритам рада Arduino програма

Променљиве i , j , k , h , f и g су целобројне променљиве, а у елементе низа $nbin$ се смештају бинарне цифре променљиве j , где је $nbin[0]$ *LSB*, а $nbin[3]$ *MSB*.

4.2.2 Програмска подршка за *Raspberry Pi*

4.2.2.1 Повезивање *AT-09* са *Raspberry Pi*-јем

Raspberry Pi узима улогу клијента у комуникацији. За страну клијента, *Qt BluetoothLowEnergy API* омогућава креирање веза ка периферним уређајима, откривање њихових сервиса и читање/упис у карактеристике *BLE* периферних уређаја [26]. Повезивање модула је извршено по узору на *Heart Rate Game* са *Qt* сајта. Датотеке *deviceinfo.h* и *deviceinfo.cpp* се убацују у пројекат као и *chess.h*, док се датотеке *devicehandler.cpp* и *devicehandler.h* модификују да одговарају овом систему [27], [26]. Назив *devicehandler* је преименован у *boardscanner*.

4.2.2.1.1 Успостављање везе

Проналазак и повезивање на периферни уређај захтева од периферних уређаја да оглашавају своје присуство и сервисе. *AT-09* има улогу периферног уређаја. За откривање периферних уређаја се користи класа *QBluetoothDeviceDiscoveryAgent*. Конструктор класе *BoardScanner* је приказан у код листингу 4-1 [27].

Код листинг 4-1 Конструктор класе *BoardScanner*

```
BoardScanner::BoardScanner(QWidget *parent):
    m_currentDevice(QBluetoothDeviceInfo()),
    foundBoardScannerService(false),
    m_control(0), m_service(0)
{
    m_deviceDiscoveryAgent = new
    QBluetoothDeviceDiscoveryAgent(this);

    connect(m_deviceDiscoveryAgent,
            SIGNAL(deviceDiscovered(const QBluetoothDeviceInfo&)),
            this, SLOT(addDevice(const QBluetoothDeviceInfo&)));
    connect(m_deviceDiscoveryAgent,
            SIGNAL(error(QBluetoothDeviceDiscoveryAgent::Error)),
            this, SLOT(deviceScanError(QBluetoothDeviceDiscoveryAgent::Error)));
    connect(m_deviceDiscoveryAgent, SIGNAL(finished()),
            this, SLOT(scanFinished()));
}
```

Конструктор инстанцира објекат класе *QBluetoothDeviceDiscoveryAgent*, *m_deviceDiscoveryAgent* и повезује његов сигнал *deviceDiscovered* са *BoardScanner* слотом *addDevice*. Параметар сигнала и слота је класа *QBluetoothDeviceInfo* која садржи податаке о *Bluetooth* уређају – име, адреса и класа уређаја. Адреса уређаја независна од платформе и протокола је дата класом *QBluetoothAddress* [26].

Јавни слот *deviceSearch* класе *BoardScanner* позива јавни слот *start* објекта *m_deviceDiscoveryAgent*. Позивом *start* се почиње са претрагом. Слот *deviceSearch* је приказан у код листингу 4-2 [27].

Код листинг 4-2 Слот *deviceSearch*

```
void BoardScanner::deviceSearch()
{
    qDeleteAll(m_devices);
    m_devices.clear();
    m_deviceDiscoveryAgent->start();
    setMessage("Scanning for devices...");
}
```

Сигнал *deviceDiscovered* се емитује када се уређај пронађе, а потом, уколико је уређај *BLE* уређај, *addDevice* сачува *QBluetoothDeviceInfo* податке о уређају на крај листе *m_devices*. Слот *addDevice* је приказан у код листингу 4-3 [27].

Код листинг 4-3 Слот addDevice

```
void BoardScanner::addDevice(const QBluetoothDeviceInfo &device)
{
    if (device.coreConfigurations() &
        QBluetoothDeviceInfo::LowEnergyCoreConfiguration) {
        qWarning() << "Discovered LE Device name: "
                    << device.name() << " Address: "
                    << device.address().toString();
        DeviceInfo *dev = new DeviceInfo(device);
        m_devices.append(dev);
    }
}
```

Када се откривање уређаја заврши емитоваће се `finished` и извршиће се `scanFinished`. Слот `scanFinished` је приказан у код листингу 4-4 [27].

Код листинг 4-4 Слот scanFinished

```
void BoardScanner::scanFinished() {
    if (m_devices.isEmpty()) {
        setMessage("No Low Energy devices found");
        emit exit();
    }
    else {
        for(QObject *obj : m_devices){
            DeviceInfo *dev = static_cast<DeviceInfo*>(obj);
            if(dev->getDevice().name() == "BT05"){
                setMessage("Bluetooth Board found.");
                connectToService(dev->getAddress());
                return;
            }
        }
        setMessage("Board not found");
        emit exit();
    }
}
```

Приликом извршавања `scanFinished`, уколико при проласку кроз листу `m_devices` уређај има име *BT05*, позива се јавни слот `connectToService` класе `BoardScanner` са адресом тог уређаја.

4.2.2.1.2 Приступ BLE уређајима – QtLowEnergyController

Слот `connectToService` је приказан у код листингу 4-5 [27].

Код листинг 4-5 Слот connectToService

```
void BoardScanner::connectToService(const QString &address){
    bool deviceFound = false;
    for (int i = 0; i < m_devices.size(); i++) {
        if (((DeviceInfo*)m_devices.at(i))->getAddress() == address ) {
            m_currentDevice.setDevice(((DeviceInfo*)m_devices.at(i))->getDevice());
            setMessage("Connecting to device...");
            deviceFound = true;
            break;
        }
    }
    if (!deviceFound) {
        setMessage("Device not found.");
        emit exit();
        return;
    }

    m_control = new QLowEnergyController(m_currentDevice.getDevice(), this);
    connect(m_control, SIGNAL(serviceDiscovered(QBluetoothUuid)),
            this, SLOT(serviceDiscovered(QBluetoothUuid)));
}
```

```

connect(m_control, SIGNAL(discoveryFinished()),
        this, SLOT(serviceScanDone()));
connect(m_control, SIGNAL(error(QLowEnergyController::Error)),
        this, SLOT(controllerError(QLowEnergyController::Error)));
connect(m_control, SIGNAL(connected()), this, SLOT(deviceConnected()));
connect(m_control, SIGNAL(disconnected()), this, SLOT(deviceDisconnected()));
m_control->connectToDevice();
}

```

Унутар слота `connectToService` се на основу пронађеног уређаја инстанцира објект `m_control` класе `QLowEnergyController`. Ова класа омогућава приступ *BLE* уређајима. Врше се следећа повезивања:

- повезивање сигнала `serviceDiscovered` објекта `m_control` и слота `serviceDiscovered` класе `BoardScanner`
- повезивање сигнала `discoveryFinished` објекта `m_control` и слота `serviceScanDone` класе `BoardScanner`
- повезивање сигнала `connected` објекта `m_control` и слота `deviceConnected` класе `BoardScanner`
- повезивање сигнала `disconnected` објекта `m_control` и слота `deviceDisconnected` класе `BoardScanner`.

Позивом `connectToDevice` објекта `m_control` врши се повезивање на удаљени *BLE* уређај. Метода не ради ништа ако контролер није у неповезаном стању. Сигнал `connected` се емитује када је веза успостављена [26]. Тада се извршава слот `deviceConnected` класе `BoardScanner`.

4.2.2.1.3 Трагање за сервисом модула AT-09

Слот `deviceConnected` је приказан у код листингу 4-6 [27].

Код листине 4-6 Слот deviceConnected

```

void BoardScanner::deviceConnected() {m_control->discoverServices();}

```

Слот `deviceConnected` покреће претрагу сервиса, позивом `discoverServices` контролера `m_controller`. Сваким открићем новог сервиса се емитује сигнал `serviceDiscovered`. Параметар овог сигнала и њему одговарајућег слота `serviceDiscovered` је *UUID* пронађеног сервиса, класе `QBluetoothUuid`. Слот `serviceDiscovered` је приказан у код листингу 4-7 [27].

Код листине 4-7 Слот serviceDiscovered

```

void BoardScanner::serviceDiscovered(const QBluetoothUuid &gatt) {
    if (gatt == QBluetoothUuid(QStringLiteral("0000ffe0-0000-1000-8000-00805f9b34fb"))) {
        setMessage("BoardScanner service discovered. Waiting for service scan to be done...");
        foundBoardScannerService = true;
    }
}

```

`BoardScanner` имплементира слот `serviceDiscovered` тако да игнорише све сервисе чији *UUID* није једнак *UUID*-ју сервиса модула AT-09 (откивеног под именом BT05). Тражени *UUID* је 0xFFE0, с тим што се пореди његова реконструисана 128-битна вредност (видети потпоглавље 2.4.3.1 ATT). Уколико се тај сервис пронађе, флег класе `BoardScanner` – `foundBoardScannerService` ће имати вредност `true`. Када се претрага заврши, контролер `m_controller` емитује сигнала `discoveryFinished`, чиме се извршава слот `serviceScanDone`. Слот `serviceScanDone` је приказан у код листингу 4-8 [27].

```

void BoardScanner::serviceScanDone()
{
    if (foundBoardScannerService) {
        setMessage("Connecting to service...");
        m_service = m_control->createServiceObject(
            QBluetoothUuid(QStringLiteral("0000ffe0-0000-1000-8000-00805f9b34fb")), this);
    }

    if (!m_service) {
        setMessage("BoardScanner Service not found.");
        emit exit();
        return;
    }

    connect(m_service, SIGNAL(stateChanged(QLowEnergyService::ServiceState)),
            this, SLOT(serviceStateChanged(QLowEnergyService::ServiceState)));
    connect(m_service,
            SIGNAL(characteristicChanged(QLowEnergyCharacteristic,QByteArray)),
            this, SLOT(updateBoardState(QLowEnergyCharacteristic,QByteArray)));
    connect(m_service, SIGNAL(descriptorWritten(QLowEnergyDescriptor,QByteArray)),
            this,
            SLOT(confirmedDescriptorWrite(QLowEnergyDescriptor,QByteArray)));

    m_service->discoverDetails();
}

```

Уколико је жељени сервис пронађен, односно уколико је `foundBoardScannerService == true`, креира се инстанца класе `QLowEnergyService` која представља сервис модула *AT-09*. Класа омогућава приступ детаљима сервиса. Њени сигнали су:

- `stateChanged` – емитује се сваки пут када се стање сервиса промени
- `characteristicChanged` – емитује се сваки пут када се вредност карактеристике промени одговарајућим догађајем на периферном уређају *AT-09*
- `descriptorWritten` – емитује се сваки пут када се вредност дескриптора успешно промени. Та промена се врши помоћу методе `writeDescriptor` [26].

Ови сигнали се повезују са слотовима класе `BoardScanner`:

- Сигнал `stateChanged` се повезује са `serviceStateChanged`
- Сигнал `characteristicChanged` се повезује са `updateBoardState`
- Сигнал `descriptorWritten` се повезује са `confirmedDescriptorWrite`.

Последња ствар која се ради у слоту `serviceScanDone` је позив методе `discoverDetails`. Њоме се иницира откривање сервиса садржаних у пронађеном сервису модула *AT-09*, карактеристика и њихових придружених дескриптора. Током претраге детаља се мења стање сервиса које на крају завршава у `ServiceDiscovered` стању, где се познају детаљи сервиса, као што су све саржане карактеристике, дескриптори и садржани сервиси. Вредности карактеристика и дескриптора се могу добити помћу класа `QLowEnergyCharacteristic` и `QLowEnergyDescriptor` [26]. По промени стања се емитује сигнал `stateChanged` и извршава `serviceStateChanged`. Слот `serviceStateChanged` је приказан у код листингу 4-9 [27].

```

void BoardScanner::serviceStateChanged(QLowEnergyService::ServiceState s)
{
    switch (s) {
        case QLowEnergyService::ServiceDiscovered:
        {
            const QLowEnergyCharacteristic bsChar = m_service->characteristic(
                QBluetoothUuid(QStringLiteral("0000ffe1-0000-1000-8000-00805f9b34fb")));

```

```

        if (!bsChar.isValid()) {
            setMessage("BS Data not found.");
            emit exit();
            break;
        }
        m_notificationDesc = bsChar.descriptor(
            QBluetoothUuid::ClientCharacteristicConfiguration);

        if (m_notificationDesc.isValid()) {
            m_service->writeDescriptor(m_notificationDesc,
                                       QByteArray::fromHex("0100"));
        }
        break;
    }
    default:
        //nothing for now
        break;
    }
}
}

```

У случају да је стање сервиса `ServiceDiscovered`, позива се `m_service` метода `characteristic` са *UUID* вредности која одговара *UUID* вредности карактеристике модула *AT-09* – `0xFFE1`. Метода ће вратити карактеристику која одговара том *UUID*-ју уколико постоји, а у супротном невалидну карактеристику [26]. Повратна вредност методе се смешта у објекат `bsChar` класе `QLowEnergyCharacteristic`.

Уколико је карактеристика валидна, у поље `m_notificationDesc` класе `BoardScanner` се смешта повратна вредност `bsChar` методе `descriptor`. Повратна вредност методе је дескриптор за *UUID* прослеђен као параметар или невалидна инстанца класе `QLowEnergyDescriptor`. Метода се позива са вредношћу набројивог типа `DescriptorType` класе `QBluetoothUuid`. Набројиви тип `DescriptorType` саржи *UUID*-јеве дескриптора [26].

`QBluetoothUuid::ClientCharacteristicConfiguration` је *UUID* који се прослеђује као параметар методи `descriptor` и стандардни *UUID* дескриптора *CCCD* (енгл. *Client Characteristic Configuration Descriptor*). Он омогућава укључивање и искључивање могућности да сервер сам иницира обавештења о карактеристици у којој се *CCCD* налази [8].

Уколико је дескриптор валидан, `m_service` позива методу `writeDescriptor`. Ова метода поставља нову вредност дескриптора *CCCD*, `0x0001`. Ово резултује захтевом за упис у модул *AT-09*, и ако је операција успешна, емитује се сигнал `descriptorWritten` и извршава слот `confirmedDescriptorWrite`. Слот `confirmedDescriptorWrite` је приказан у код листингу 4-10 [27].

Код листинг 4-10 Слот `confirmedDescriptorWrite`

```

void BoardScanner::confirmedDescriptorWrite(const QLowEnergyDescriptor &d,
                                           const QByteArray &value){
    if (d.isValid() && d == m_notificationDesc && value ==
        QByteArray::fromHex("0100")){
        setMessage("Bluetooth Chess Board connected!");
        emit connected();
    }
    if (d.isValid() && d == m_notificationDesc && value ==
        QByteArray::fromHex("0000")) {
        //disabled notifications -> assume disconnect intent
        m_control->disconnectFromDevice();
    }
}

```

`confirmedDescriptorWrite` препознаје да је уписана вредност у дескриптор *CCCD* `0x0001`, чиме су укључена сервер-иницирана обавештења о карактеристици модула *AT-*

09. Овде се емитује BoardScanner сигнал connected. У том тренутку апликација на Raspberry Pi рачунару од AT-09 добија нова стања табле.

4.2.2.1.4 Интеракција са сервером

Када се вредност карактеристике модула AT-09 промени, емитује се сигнал characteristicChanged и извршава се слот updateBoardState. Слот од низа бајтова које добије као параметар value реконструише низ new_state који представља стање табле представљено као и на Arduino страни. Након реконструкције се емитује сигнал boardState. Слот updateBoardState је приказан у код листингу 4-11 [27].

Код листинг 4-11 Слот updateBoardState

```
void BoardScanner::updateBoardState(const QLowEnergyCharacteristic &c,
                                   const QByteArray &value)
{
    // ignore any other characteristic change -> shouldn't really happen though
    if (c.uuid() != QBluetoothUuid(QStringLiteral("0000ffe1-0000-1000-8000-00805f9b34fb")))
        return;

    if(value.size() == 8){
        int idx = 0;

        for(int i = 0; i < 8; ++i){
            uint8_t byte = static_cast<uint8_t>(value[i]);
            for(int j = 0; j < 8; ++j){
                new_state[idx++] = (byte >> j) & 0x01;
            }
        }
        emit boardState(new_state);
    }
}
```

4.2.2.2 Графички кориснички интерфејс

Графички кориснички интерфејс је дизајниран по узору на познате шаховске сајтове попут chess.com и lichess.org. Овим се добија на предвидљивости – корисник је вероватно већ навикао на позицију појединачних елемената са којима интерагује на екрану и моћи ће на лак и интуитиван начин да интерагује са њима. На слици 4-12 је приказан графички кориснички интерфејс сајта lichess.org [28].



Слика 4-12 lichess.org графички кориснички интерфејс [28]

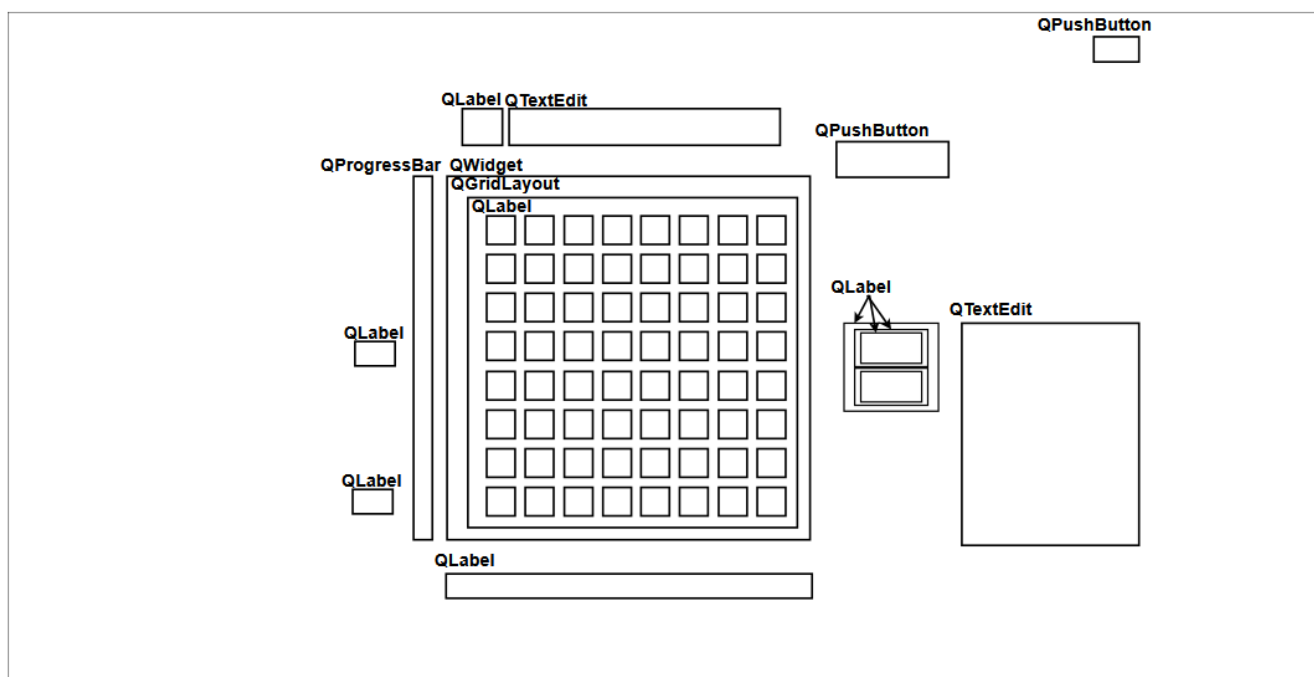
Табла је највећи елемент. Она се поставља на центар екрана. Око ње се налазе пропратна поља:

- евалуациони бар и лабела
- историја потеза
- предложен најбољи потез
- могућност приказа/отклањања евалуационог бара и лабеле.

Додатно, поља која се не виде у примеру са слике 4-12, а користе се у креираном графичком корисничком интерфејсу су:

- сатови који приказују преостала времена за играче
- статусна лабела
- *Bluetooth* обавештења
- дугме за излаз из апликације.

На слици 4-13 је приказан распоред објеката корисничког интерфејса.



Слика 4-13 Распоред објеката корисничког интерфејса

Објекти су `QWidget`-и и/или изведене класе. `QLabel` се користи за већину текстуалних објеката:

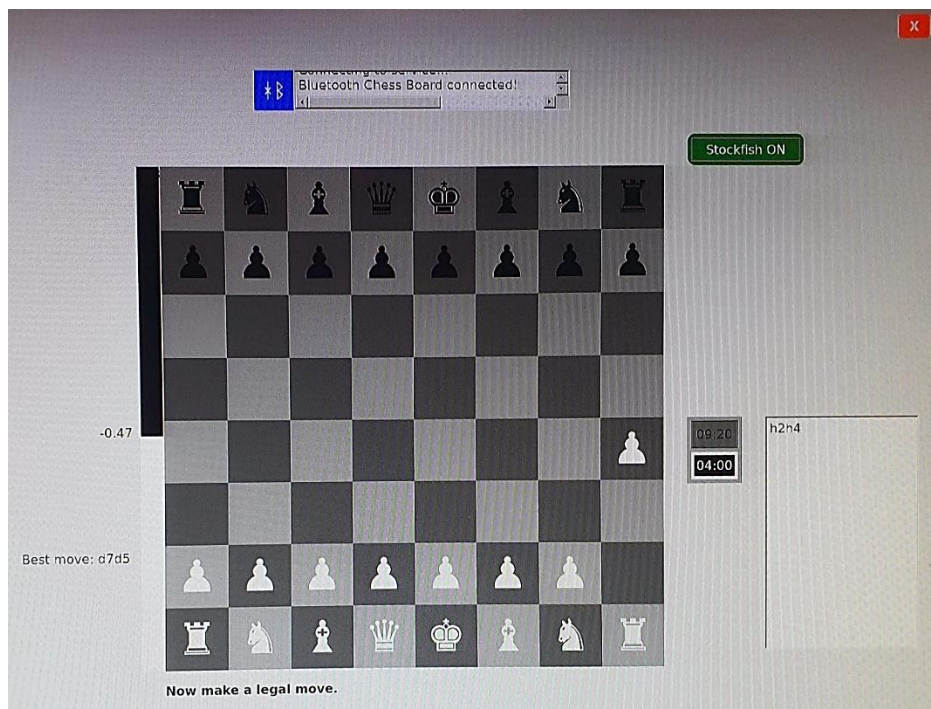
- поља табле садрже фигуре фигуре су само *Unicode* симболи (♔, ♚, ♜, ♞ ...)
- евалуациона лабела за репрезентацију евалуације бројем
- лабела за најбољи потез
- шаховски сатови су поља са текстуалним приказом остатка времена
- *Bluetooth* лого поред *Bluetooth* обавештења – *Unicode* симболи (* и ™).

Табла је један `QWidget` унутар којег је `QGridLayout` који организује `QWidget`-е поља табле у матрицу са 8 врста и 8 колона. Историја потеза и *Bluetooth* обавештења су инстанце класе `QTextEdit`. Дугме за излаз из апликације и приказ/отклањање евалуације позиције су инстанце класе `QPushButton`. Евалуациони бар је `QProgressBar`.

Са гледишта `QWidget` објеката интерфејса, класа `Dialog` инстанцира ове објекте у свом конструктору, повезује сигнале `clicked` класе `QPushButton`. За дугме за излаз `clicked` се повезује на излаз из апликације и прекид везе, а дугме за приказ/отклањање евалуације позиције на ламбда функцију која примењује `show/hide` методе на елементе

које треба сакрити – лабелу најбољег потеза, лабелу за евалуацију позиције и евалуациони бар.

На слици 4-14 је приказан изглед графичког корисничког интерфејса.



Слика 4-14 Изглед графичког корисничког интерфејса

Приликом избора временског формата на почетку партије и приликом промоције пешака појављује се мени за избор акције имплементиран класом `QMenu`. На слици 4-15 су приказани менији.



Слика 4-15 Мени за избор временског формата (лево), промоције белих фигура (средина) и промоције црних фигура (десно)

За мерење преосталог времена које је на располагању играчу користи се инстанца класе `QTimer`, која се креира у конструктору класе `Dialog`. Корисник у менију приказаном на слици 4-15 бира временски формат за играче. Доступне су вредности од 5, 10 и 30 min, као и опција играња без временског ограничења. Уколико се изабере неко временско ограничење, изабрана вредност се у секундама уписује у променљиве `timeWhite` и `timeBlack`, које представљају преостало време играча са белим и црним фигурама, редом.

Након избора времена, `QTimer` се покреће са периодом од једне секунде, а његов `timeout` сигнал је повезан са `Dialog` слотом `update_chess_clocks`. При сваком истеку периода, умањује се време играча који је на реду. Преостало време се приказује помоћу одговарајућих `QLabel` објеката, приказаних са десне стране табле са слике 4-13 и са слике 4-14. `QTimer` се зауставља када се утврди завршетак партије или када истекне време једном од играча.

4.2.2.3 Обрада новог стања табле

Класа `Dialog` у свом конструктору креира инстанцу `scanner` класе `BoardScanner`. За даљи развој софтвера, класа `BoardScanner` се посматра само преко јавних слотова `deviceSearch` и `disconnectService` и сигнала које емитује – `messageChanged`, `connected`, `boardState` и `exit`. Слот `deviceSearch` и сигнали `connected` и `boardState` су објашњени у потпоглављу 4.2.2.1 Повезивање *AT-09* са *Raspberry Pi*-јем. Слотом `disconnectService` се онемогућавају сервер-иницирана обавештења уколико су претходно била укључена, а у супротном се преко контролера прекида веза са удаљеним уређајем. Сигнал `exit` се емитује у случају отказа или прекида везе са удаљеним уређајем. Обавештења о *BLE* вези која се записују изнад табле у корисничком интерфејсу, су поруке које поставља `scanner`. Он мења своје поље `m_info` и о новим вредностима поља обавештава емитовањем сигнала `messageChanged`.

Након креирања инстанце `scanner`, у конструктору класе `Dialog` врши се повезивање сигнала `scanner`-а са слотовима класе `Dialog`. Слот `moveMaker` се повезује са сигналом `boardState`. `Dialog` садржи три поља помоћу којих прати тренутно стање на табли – инстанца класе `chess::Board`, инстанца класе `chess::Move` и низ јединица и нула `boardState`, који пореди своје вредности са пристиглим `new_state` параметром сигнала `boardState`.

Поље `boardState` се може посматрати као низ тренутне заокупираности поља фигурама, чије конкретне вредности носе исто значење као и код низова `new_state` у *Arduino* програму и `new_state` у параметру сигнала `boardState`.

Улоге слота `moveMaker` су:

- детекција померања фигура
- провера легалности потеза
- руковање рокадом и промоцијом пешака
- извршавање и евиденција легалних потеза
- руковање нелегалним потезима
- освежавање интерфејса.

Слот читава ново стање пристигло од `scanner`-а и региструје где је фигура подигнута а где спуштена. Ефективно, један потез чини акција подизања и спуштања фигуре. Користећи `chess.h` генерише се листа свих легалних потеза у датој позицији и проверава да ли је одиграни потез међу њима.

Уколико се детектује промоција пешака, отвара се искачући мени приказан на слици 4-15. У њему корисник бира у коју фигуру жели да промовише пешака. Рокада се детектује као секвенца два потеза – потез краљем и потом потез топом. Када се детектује први део рокаде, једина следећа дозвољена акција је наставак рокаде потезом топом. Ова два потеза се за потребе коришћења библиотеке `chess.h` потом конвертују у њој препознатљиву нотацију рокаде, померање краља са свог иницијалног поља на поље топа.

Када се потврди да је потез валидан, он се дописује у *UCI* формату у поље са историјом потеза графичког корисничког система. Поље класе `Board` се ажурира позивом методе `makeMove`. Потом се проверава статус партије – да ли је наступио шах-мат, пат и сл. У

случају краја се зауставља сат реализован класом `QTimer`. На овом месту се позиција шаље у облику *FEN* стринга *Stockfish* програму ради анализе позиције. Мења се играч који је на потезу и пребацује се сат. Ове промене стања табле се приказују на табли на екрану освежавањем приказа поља методом `refreshField`.

Уколико је одигран нелегалан потез поставља се флег `illegal` на вредност `true` уз поруку упозорења у инстанци класе `QLabel` која се на екрану приказује испод табле. У том стању `moveMaker` пореди тренутно стање табле са последњом легалном позицијом и црвеном бојом означава различита поља све док играч не врати фигуре у исправно стање.

4.2.2.4 Комуникација са *Stockfish* програмом

Stockfish програму се задају команде преко његовог стандардног улаза (*stdin*) и чека се *Stockfish* излаз, који се појављује на његовом стандардном излазу (*stdout*). У ову сврху се користи класа `QProcess`. Она се користи да покрене екстерне програме и да комуницира са њима. Процес се покреће методом `start`. Писање на стандардни улаз процеса се врши методом `write`, а читање стандардног излаза се врши методом `read` или `readLine`. Његов сигнал `readyReadStandardOutput` се емитује када је процес обезбедио нове податке кроз стандардни излаз [29].

У конструктору класе `Dialog` се инстанцира `QProcess` објекат за *Stockfish* програм. Његов сигнал `readyReadStandardOutput` се повезује са `Dialog` слотом `parseStockfish`. Када `scanner` емитује сигнал `connected`, у слоту који одговара том сигналу, `onConnected`, се поставља путања до шаховског програма. У овом пројекту локална путања је `../engine/stockfish`. Потом се процес покреће методом `start`. Методом `write` се шаљу *UCI* команде `uci` како би *Stockfish* користио *UCI*, и потом `isready`, за синхронизацију *Stockfish* програма и апликације.

На одговор `readyok` слот `parseStockfish` позива искачући мени за избор временског формата, преузима *FEN* тренутне позиције у пољу класе `Board` и шаље команде `position fen <fenstring>` и `go movetime 150`. Првом командом се позиција поставља на ону у пољу класе `Board`. Другом се почиње са рачунањем на постављеној позицији уз ограничење претраге најбољег потеза на приближно 150 ms.

Унутар слота `moveMaker`, приликом сваког одиграног легалног потеза, претходна анализа са прекида командом `stop`. Потом се шаље новонастала позиција у *FEN* стрингу и поново се покреће анализа. Док *Stockfish* анализира позицију, слот `parseStockfish` у петљи чита линије излаза.

Линије које почињу са `info` садрже тренутне резултате анализе. Ове линије садрже доста додатних поља са информацијама о току анализе. За приказ најбољег потеза, бројчане и визуелне процене позиције су од значаја само следећа поља:

1. поље облика `pv <move1> ... <movei>` је најбоља пронађена линија;
2. поље `score` које прате додатна поља, али од интереса су:
 - 2.1. `cp <x>` – процена позиције са гледишта стране за коју *Stockfish* програм тренутно тражи најбољи потез, изражен у стотим деловима пешака;
 - 2.2. `mate <y>` – мат у `y` потеза за страну за коју *Stockfish* програм тренутно тражи најбољи потез.

Уколико је страна за коју *Stockfish* програм тренутно тражи најбољи потез у лошијој позицији, вредност `x` у пољу `cp <x>` ће бити негативна. Уколико је страна за коју *Stockfish* програм тренутно тражи најбољи потез матирана, вредност `y` у пољу `mate <y>` ће бити негативна [30].

Када таква линија буде доступна на излазу, за приказ следећег најбољег потеза `parseStockfish` слот мора само да издвоји први потез, `<move1>`, из поља `pv <move1> ... <movei>`. `parseStockfish` слот парсира целобројну вредност `x` из `cp <x>` уколико је `info` линија са тим пољем. На корисничком интерфејсу се процена излаже из аспекта

играча са белим фигурама. Због тога се узима негирана вредност поља x ако је потребно приказати бројчану евалуацију, а играч са црним фигурама је на потезу. На тај начин се процена у корисничком интерфејсу приказује из исте перспективе независно од тога који играч је на потезу, чиме се избегава промена предзнака процене након сваког новог потеза. Пре него што се вредност прикаже на екрану, дели се са 100. На овај начин се процена изражена у стотим деловима пешака изражава у пешацима. Са становишта мата, текстуална евалуација увек има облик M_y , у означава да је мат у y потеза. Остало је још мапирати евалуацију позиције изражену у стотим деловима пешака у вредности које могу бити представљене на евалуационом бару – $[0, 100]$. Показало се да је логистичка функција добра апроксимација [31]. Логистичка функција $p(cp)$ исказана једначином 2.

$$p(cp) = \frac{1}{1 + e^{-\frac{cp}{A}}} \quad (2)$$

Параметар A у једначини 2 одређује стрмину логистичке функције. Овде, искључиво у сврху визуелног приказа евалуације, се овај параметар интерпретира као осетљивост траке. У пројекту је изабрано $A = 400$. Вредности $p(cp)$ су из опсега $(0, 1)$, стога се множе са 100 како би се резултујућа вредност – $(0, 100)$ могла унети као нова вредност евалуационог бара. Матови не прате логистичку функцију већ, уколико је на табли мат, вредност бара постаје 0 или 100 у зависности од тога ко је потенцијално матиран.

4.3 Систем у крајњем стадијуму израде пројекта

На слици 4-16 је приказан систем у крајњем стадијуму израде пројекта.



Слика 4-16 Систем у крајњем стадијуму израде пројекта

5. ЗАКЉУЧАК

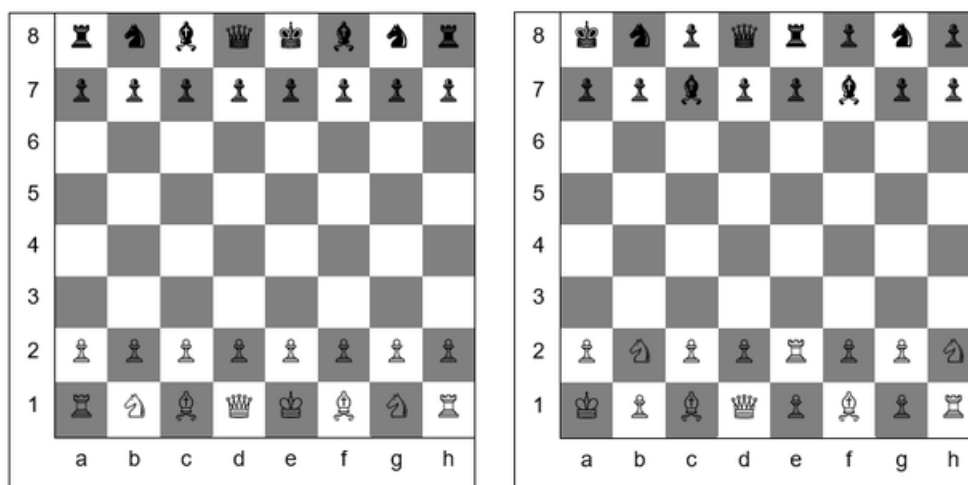
Систем пројектован у оквиру овог рада омогућава једноставну и интуитивну интеракцију са корисницима, односно играчима шаха. Са физичког становишта, кориснику су на располагању реална табла, тастатура, миш и монитор као средства за интеракцију са системом. Са друге стране, графички кориснички интерфејс на монитору пружа начин управљања и приказа података који одговара уобичајеним шаховским софтверским решењима.

У модерно време се анализа партија и учење теорије све више врши електронски. Овај уређај се може тако посматрати – као алат за вежбу против шаховског програма или као шаховска табла за игру између два играча. Интеграцијом програма *Stockfish* омогућени су приказ евалуације позиције и предлагање најбољег пронађеног потеза.

Увођењем бежичне комуникације систем је добио могућност једноставне размене података између физичког дела шаховске табле и *Raspberry Pi* рачунара. Оваква архитектура представља основу за даље повезивање система са другим уређајима и сервисима и отвара могућности за примену концепата интернета ствари (енгл. *Internet of Things – IoT*).

Дакле, могућност даљег развоја представља умрежавање система са другим шаховским системима или шаховским сајтовима, чиме би се омигућило чување и размена одиграних партија, као и играње између корисника који се налазе на различитим локацијама.

Са становишта детекције положаја фигура, у овом раду су примењени дигитални *Hall* сензори, првенствено због своје ниске цене. Сваком пољу шаховске табле је додељен по један сензор. Постоје и друга решења за детекцију положаја фигура, као што су системи засновани на камерама или *RFID* (енгл. *Radio-Frequency Identification*) технологији. Њиховом применом могле би се остварити додатне могућности, на пример препознавање типа конкретне фигуре на пољу табле. За тренутни систем, позиције приказане на слици 5-1 представљају идентично стање.



Слика 5-1 Две позиције једнаке окупације поља фигурама

Ипак, овим би се архитектура система у већој мери усмерила ка сложенијој обради података на софтверској страни. Реализованим поступком показано је да се коришћењем релативно једноставних хардверских компоненти може реализовати функционална шаховска табла која испуњава основне захтеве постављене у раду.

Тестирање система спроведено је на прототипу реализованом на протоборду. Једно од могућих хардверских унапређења оредстављало би пројектовање и израда штампане плоче (енгл. *Printed Circuit Board – PCB*). На тај начин би се могла повећати механичка и електрична робусност система уз задржавање основне функционалности реализованог решења.

6. ЛИТЕРАТУРА

- [1] д. И. Мезеи, Рачунарска електроника - практикум лабораторијских вежби, Нови Сад: Факултет техничких наука, 2018.
- [2] R. P. Ltd, „Raspberry Pi,“ Raspberry Pi Ltd, (На мрежи). Доступно на: <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>. (Последњи приступ 7 септембар 2026).
- [3] I. Allegro MicroSystems, „A3144 Datasheet: Sensitive Hall-Effect Switches for High-Temperature Operation,“ Allegro MicroSystems, Inc, (На мрежи). Доступно на: <https://www.alldatasheet.com/html-pdf/1747660/ALLEGRO/A3144/687/2/A3144.html>. (Последњи приступ 7 септембар 2026).
- [4] д. И. Мезеи, Основи микропроцесорских и микроконтролерских система, Нови Сад: Факултет техничких наука, 2022.
- [5] М. Л. В. Н. И. М. Душан Бортник, „Серијски порт и функције за серијску комуникацију,“ у *Развој софтвера за ембедед системе*, Инфоелектроника, 2023, pp. 85-90.
- [6] R. Heydon, „Chapter 1. What Is Bluetooth Low Energy?,“ у *Bluetooth Low Energy Developer's Handbook*, 2012.
- [7] C. C. A. & R. D. Kevin Townsend, „Chapter 1. Introduction & Chapter 2. Protocol Basics,“ у *Getting Started with Bluetooth Low Energy*, 2014.
- [8] C. C. A. & R. D. Kevin Townsend, „Chapter 4. GATT (Services and Characteristics),“ у *Getting Started with Bluetooth Low Energy*, 2014.
- [9] C. C. A. & R. D. Kevin Townsend, „Chapter 3. GAP (Advertising and Connections),“ у *Getting Started with Bluetooth Low Energy*, 2014.
- [10] T. Instruments, „BLE-STACK Driver or library,“ Texas Instruments, (На мрежи). Доступно на: https://www.ti.com/lit/ug/swru271i/swru271i.pdf?ts=1789255372336&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FCC2541. (Последњи приступ 13 септембар 2026).
- [11] Bolutek, „BLE-CC41-A Bluetooth Module Specification,“ Bolutek, (На мрежи). Доступно на: <https://device.report/m/2aef2229830313a8a39361198439ef8a92b76d9bc3ee71cccb3e7f134e52f74a.pdf>. (Последњи приступ 13 септембар 2026).
- [12] Bolutek, „BLE-CC41-A Bluetooth Module AT Commands,“ Bolutek, (На мрежи). Доступно на: <https://device.report/m/e5c44b6d15057164a8875389d1bf11a99892802a21006db068443dd1ac34bc32.pdf>. (Последњи приступ 13 септембар 2026).
- [13] A. Yavilevich, „HM-10 or CC41-A module? Automatic Arduino BLE module identification,“ (На мрежи). Доступно на: <https://blog.yavilevich.com/2016/12/hm-10-or-cc41-a-module-automatic-arduino-ble-module-identification/>. (Последњи приступ 13 септембар 2026).
- [14] E. Wiki, „ElectroDragon Wiki,“ (На мрежи). Доступно на: <https://w.electrodragon.com/w/CC2541>. (Последњи приступ 13 септембар 2026).

- [15] [www.ictransistors.com](https://www.ictransistors.com/bluetooth-module/at-09-4-0-bluetooth-module-ble-serial-port.html), „AT-09 4.0 Bluetooth Module Ble Serial Port With Backplane,“ (На мрежи). Доступно на: <https://www.ictransistors.com/bluetooth-module/at-09-4-0-bluetooth-module-ble-serial-port.html>. (Последњи приступ 13 септембар 2026).
- [16] M. Currey, „Bluetooth Modules,“ (На мрежи). Доступно на: <https://www.martyncurrey.com/bluetooth-modules/#AT-09>. (Последњи приступ 13 септембар 2026).
- [17] [chess.com](https://www.chess.com/terms/fen-chess), „Forsyth-Edwards Notation (FEN),“ (На мрежи). Доступно на: <https://www.chess.com/terms/fen-chess>. (Последњи приступ 16 септембар 2026).
- [18] Wikipedia, „Algebraic notation (chess),“ (На мрежи). Доступно на: [https://en.wikipedia.org/wiki/Algebraic_notation_\(chess\)#Long_algebraic_notation](https://en.wikipedia.org/wiki/Algebraic_notation_(chess)#Long_algebraic_notation). (Последњи приступ 16 септембар 2026).
- [19] Disservin, „chess-library - Documentation,“ (На мрежи). Доступно на: <https://disservin.github.io/chess-library>. (Последњи приступ 16 септембар 2026).
- [20] Disservin, „chess-library,“ (На мрежи). Доступно на: <https://github.com/Disservin/chess-library/tree/master/src>. (Последњи приступ 16 септембар 2026).
- [21] T. S. developers, „Stockfish Docs,“ (На мрежи). Доступно на: <https://official-stockfish.github.io/docs/stockfish-wiki/Home.html>.
- [22] Wikipedia, „Universal Chess Interface,“ (На мрежи). Доступно на: https://en.wikipedia.org/wiki/Universal_Chess_Interface. (Последњи приступ 16 септембар 2026).
- [23] М. Лукић, „rszes - biblioteka,“ (На мрежи). Доступно на: <https://github.com/rszes/biblioteka/tree/main/usart>. (Последњи приступ 16 септембар 2026).
- [24] Q. Group, „Signals & Slots,“ (На мрежи). Доступно на: <https://doc.qt.io/archives/qt-5.15/signalsandslots.html>. (Последњи приступ 17 септембар 2026).
- [25] T. Instruments, „CD74HCx4067 High-Speed CMOS Logic 16-Channel Analog Multiplexer and Demultiplexer,“ (На мрежи). Доступно на: <https://www.ti.com/lit/ds/schs209d/schs209d.pdf?ts=1774829718968>. (Последњи приступ 17 септембар 2026).
- [26] Q. Group, „Bluetooth Low Energy Overview,“ (На мрежи). Доступно на: <https://doc.qt.io/qt-6/qtbluetooth-le-overview.html>. (Последњи приступ 19 септембар 2026).
- [27] G. Qt, „Bluetooth Low Energy Heart Rate Game,“ (На мрежи). Доступно на: <https://code.qt.io/cgit/qt/qtconnectivity.git/tree/examples/bluetooth/heart-rate-game?h=5.10>. (Последњи приступ 19 септембар 2026).
- [28] (На мрежи). Доступно на: <https://lichess.org/>. (Последњи приступ 21 септембар 2026).
- [29] D. Qt Group, „QProcess Class,“ (На мрежи). Доступно на: <https://doc.qt.io/qt-6/qprocess.html>. (Последњи приступ 21 септембар 2026).
- [30] aliostad, „stockfish - Description of the universal chess interface (UCI),“ (На мрежи). Доступно на: <https://gist.github.com/aliostad/f4470274f39d29b788c1b09519e67372>. (Последњи приступ 21 септембар 2026).

[31] official-stockfish, „WDL_model,“ (На мрежи). Доступно на: https://github.com/official-stockfish/WDL_model. (Последњи приступ 21 септембар 2026).